ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



TRƯƠNG THẾ DŨNG

TÌM KIẾM CỤC BỘ ĐƯỢC HƯỚNG DẪN BỞI MẠNG NƠ-RON ĐỒ THỊ CHO BÀI TOÁN CÁI TÚI HỢP TẬP

LUẬN VĂN THẠC SĨ [TÊN NGÀNH]

HÀ NỘI – 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

TRƯƠNG THẾ DŨNG

TÌM KIẾM CỤC BỘ ĐƯỢC HƯỚNG DẪN BỞI MẠNG NƠ-RON ĐỒ THỊ CHO BÀI TOÁN CÁI TÚI HỢP TẬP

Ngành: [Tên ngành]
Chuyên ngành: [Tên chuyên ngành]
Mã số: [Mã số]

LUẬN VĂN THẠC SĨ [TÊN NGÀNH]

CÁN BỘ HƯỚNG DẪN KHOA HỌC:
[Học hàm, học vị, họ và tên cán bộ hướng dẫn]

HÀ NỘI – 2026

LỜI CAM ĐOAN

Tôi xin cam đoan luận văn "*Tìm kiếm cục bộ được hướng dẫn bởi mạng nơ-ron đồ thị cho bài toán cái túi hợp tập*" là công trình nghiên cứu của riêng tôi, được thực hiện dưới sự hướng dẫn khoa học của [Học hàm, học vị, họ và tên cán bộ hướng dẫn]. Các kết quả thực nghiệm trình bày trong luận văn do tôi trực tiếp cài đặt, chạy và ghi nhận; mã nguồn và dữ liệu kết quả được lưu trữ cùng luận văn. Các số liệu của những công trình khác được trích dẫn đầy đủ từ các tài liệu đã công bố.

Tôi xin chịu hoàn toàn trách nhiệm về tính trung thực của các nội dung trong luận văn này.

Hà Nội, ngày ... tháng ... năm 2026

Học viên

Trương Thế Dũng

LỜI CẢM ƠN

Tôi xin bày tỏ lòng biết ơn sâu sắc tới [Học hàm, học vị, họ và tên cán bộ hướng dẫn], người đã tận tình hướng dẫn, định hướng và góp ý cho tôi trong suốt quá trình thực hiện luận văn. Những trao đổi về tìm kiếm cục bộ và tối ưu tổ hợp đã giúp tôi nhìn bài toán theo góc độ của người làm tối ưu, từ đó đặt phần học máy vào đúng chỗ trong một thuật toán tìm kiếm.

Tôi xin cảm ơn các thầy cô giáo Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội đã truyền đạt kiến thức và tạo điều kiện thuận lợi cho tôi trong quá trình học tập và nghiên cứu.

Cuối cùng, tôi xin cảm ơn gia đình và bạn bè đã luôn động viên, ủng hộ tôi trong suốt thời gian qua.

TÓM TẮT

Tóm tắt: Bài toán cái túi hợp tập (SUKP) là một biến thể NP-khó của bài toán cái túi, trong đó mỗi item là một tập element có trọng số và trọng số của element dùng chung chỉ được tính một lần. Các thuật toán tìm kiếm cục bộ tốt nhất cho SUKP, như E2LS, dựa vào một luật viết tay là tỉ số giá trị trên trọng số để quyết định thêm item nào. Luận văn này giữ nguyên khung E2LS và chỉ thay đúng một thành phần, bước xếp hạng ứng viên của toán tử ADD*, bằng một hàm được học. Luận văn cài đặt lại E2LS bằng C++ với giao diện Python cho phép thay hàm xếp hạng, thiết kế một mạng nơ-ron đồ thị trên đồ thị hai phía item–element với số tham số không phụ thuộc kích thước instance, và huấn luyện hàm xếp hạng theo hai cách: học tăng cường bằng DQN trên từng instance, và học bắt chước từ nhãn tối ưu được tính bằng tìm kiếm vét cạn trên các instance sinh ngẫu nhiên. Bộ xếp hạng học bắt chước được tham số hóa như một hiệu chỉnh của luật tỉ số, nên khi chưa huấn luyện nó trùng với thuật toán gốc. Trên các instance kiểm định chưa gặp, bộ xếp hạng nâng tỉ lệ chọn đúng ứng viên tối ưu từ 60.9% lên 71.3% và giảm regret từ 0.71% xuống 0.44%. Với một lần chạy 30 giây, bản cài đặt E2LS đạt lời giải tốt nhất đã biết trên 17/30 instance của Set I và có khoảng cách trung bình 1.45% trên Set II. Luận văn cũng phân tích chi phí suy luận của việc xếp hạng bằng mạng tại mỗi nút tìm kiếm và đề xuất quy trình đánh giá cùng số bước và cùng thời gian cho các hàm xếp hạng học được.

Từ khóa: bài toán cái túi hợp tập, tìm kiếm cục bộ, học tăng cường, mạng nơ-ron đồ thị, học máy cho tối ưu tổ hợp.

ABSTRACT

Abstract: The Set-Union Knapsack Problem (SUKP) is an NP-hard variant of the knapsack problem in which each item is a set of weighted elements and a shared element is paid only once. The best local search algorithms for SUKP, such as E2LS, rely on a hand-written value-weight ratio to decide which item to add. This thesis keeps the E2LS framework unchanged and replaces exactly one component, the candidate ranking inside the ADD* operator, with a learned function. We reimplement E2LS in C++ with a Python interface that accepts any ranking function, design a graph neural network over the item–element bipartite graph whose parameter count does not depend on the instance size, and train the ranking function in two ways: per-instance deep Q-learning, and imitation of optimal labels computed by exhaustive search on randomly generated instances. The imitation ranker is parameterised as a correction to the ratio rule, so an untrained model reproduces the original algorithm. On unseen validation instances, the learned ranker raises the rate of picking an optimal candidate from 60.9% to 71.3% and lowers the regret from 0.71% to 0.44%. With a single 30-second run, the reimplemented E2LS matches the best-known solution on 17 of the 30 Set I instances and is within 1.45%

of it on average on Set II. We also analyse the inference cost of ranking with the network at every search node, and propose an equal-steps and equal-time protocol for evaluating learned rankers on the benchmark.

Keywords: set-union knapsack problem, local search, reinforcement learning, graph neural network, machine learning for combinatorial optimization.

Mục lục

Chương 1. Tổng quan **1**

1.1 Đặt vấn đề 1

1.2 Bài toán cái túi hợp tập 2

1.3 Mục tiêu và phạm vi nghiên cứu 2

1.4 Đóng góp của luận văn 3

1.5 Cấu trúc luận văn 3

Chương 2. Cơ sở lý thuyết và các nghiên cứu liên quan **5**

2.1 Bài toán cái túi hợp tập 5

2.1.1 Định nghĩa hình thức 5

2.1.2 Mô hình quy hoạch nguyên 0-1 6

2.1.3 Độ phức tạp và các bài toán liên quan 6

2.1.4 Bộ dữ liệu chuẩn 6

2.2 Các thuật toán giải SUKP 7

2.2.1 Thuật toán chính xác và xấp xỉ 7

2.2.2 Thuật toán dựa trên quần thể 7

2.2.3 Thuật toán tìm kiếm cục bộ 7

2.3 Tìm kiếm cục bộ và tìm kiếm tabu 8

2.4 Học tăng cường 9

2.4.1 Quá trình quyết định Markov 9

2.4.2 Q-learning 9

2.4.3 Mạng Q sâu 9

2.5 Mạng nơ-ron đồ thị 10

2.6 Học máy cho tối ưu tổ hợp 10

Chương 3. Phương pháp đề xuất **12**

3.1 Kiến trúc tổng thể . . . 12
3.2 Thuật toán E2LS và bản cài đặt C++ . . . 13
3.2.1 Trọng số tăng thêm và tỉ số giá trị trên trọng số . . . 13
3.2.2 Khởi tạo tham lam ngẫu nhiên . . . 13
3.2.3 Bước tìm kiếm cục bộ và toán tử ADD* . . . 14
3.2.4 Cài đặt bằng C++ và giao diện Python . . . 15
3.3 Điểm cắm học máy: xếp hạng ứng viên của ADD* . . . 15
3.4 Mạng nơ-ron đồ thị trên đồ thị hai phía item–element . . . 16
3.4.1 Biểu diễn đồ thị . . . 16
3.4.2 Truyền thông điệp . . . 16
3.5 Phương án 1: DQN huấn luyện trên từng instance . . . 17
3.5.1 Mô hình hóa quá trình quyết định Markov . . . 17
3.5.2 Lời giải khởi đầu bằng giải thuật di truyền . . . 18
3.5.3 Huấn luyện . . . 19
3.5.4 Sử dụng trong ADD* . . . 19
3.6 Phương án 2: bộ xếp hạng học bắt chước tìm kiếm vét cạn . . . 19
3.6.1 Sinh nhãn bằng tìm kiếm vét cạn . . . 20
3.6.2 Mô hình và hàm mất mát . . . 20
3.7 So sánh hai phương án . . . 21

Chương 4. Thực nghiệm và đánh giá **22**
4.1 Thiết lập thực nghiệm . . . 22
4.2 Kết quả của bộ giải E2LS cài đặt lại . . . 23
4.3 Hành vi tìm kiếm của bộ giải . . . 25
4.4 Kết quả huấn luyện bộ xếp hạng . . . 26
4.5 Chi phí suy luận . . . 27
4.6 Đánh giá các hàm xếp hạng học được trên bộ chuẩn . . . 27
4.7 Thảo luận . . . 28

Chương 5. Kết luận và hướng phát triển **29**
5.1 Kết luận . . . 29
5.2 Hạn chế . . . 30
5.3 Hướng phát triển . . . 30

Tài liệu tham khảo **32**

Phụ lục A. Hướng dẫn tái lập kết quả **35**

A.1 Biên dịch bộ giải C++ 35

A.2 Huấn luyện bộ xếp hạng 35

A.3 Huấn luyện DQN trên một instance 35

A.4 Đánh giá trên 60 instance chuẩn 35

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Ký hiệu	**Ý nghĩa**
SUKP	Bài toán cái túi hợp tập (Set-Union Knapsack Problem)
BMCP	Bài toán phủ cực đại có ngân sách (Budgeted Maximum Coverage Problem)
E2LS	Thuật toán tìm kiếm cục bộ hiệu quả của Zhu và Luo (Efficient and Effective Local Search)
ADD*	Toán tử thêm lại nhiều phần tử theo cây rẽ nhánh trong E2LS
RL	Học tăng cường (Reinforcement Learning)
MDP	Quá trình quyết định Markov (Markov Decision Process)
DQN	Mạng Q sâu (Deep Q-Network)
GNN	Mạng nơ-ron đồ thị (Graph Neural Network)
GA	Giải thuật di truyền (Genetic Algorithm)
BKS	Lời giải tốt nhất đã biết (Best-Known Solution)
m, n	Số item và số element của một instance
p_i, w_j, C	Lợi nhuận của item i, trọng số của element j, sức chứa của túi
U_i	Tập element mà item i phủ
$P(S)$, $W(S)$	Tổng lợi nhuận và tổng trọng số (hợp tập) của lời giải S
$AW(S, i)$	Trọng số tăng thêm khi thêm i vào S (hoặc giải phóng khi bỏ i khỏi S)
$R(S, i)$	Tỉ số giá trị trên trọng số $p_i/AW(S, i)$
FC	Tập ứng viên khả thi tại một nút ADD*
r_{num}, a_{num}	Số item được thử bỏ và số ứng viên được rẽ nhánh tại mỗi nút ADD*
$Q_\theta(s, a)$	Hàm giá trị hành động được xấp xỉ bởi mạng nơ-ron tham số θ
γ	Hệ số chiết khấu

Danh sách hình vẽ

1.1 Một instance SUKP nhỏ dưới dạng đồ thị hai phía item–element. Item được chọn và các element chúng phủ được tô màu xanh; element e_2 dùng chung nên chỉ tính trọng số một lần. 2

3.1 Quy trình hai pha: huấn luyện mạng ngoại tuyến, sau đó dùng mạng để xếp hạng ứng viên trong ADD*. 12

3.2 Cây tìm kiếm của một bước E2LS với $r_{num} = a_{num} = 2$. v_1, v_2: hai item có tỉ số nhỏ nhất bị thử bỏ; a_1, a_2: hai ứng viên được xếp hạng cao nhất tại mỗi nút ADD*. Một nhánh kết thúc khi không còn item nào vừa túi. . . . 14

3.3 Một vòng truyền thông điệp trên đồ thị hai phía: (1) item gửi tới element, (2) element gửi tới item, (3) tổng hợp lên nút toàn cục g. Item màu xanh lá là item đã được chọn. 17

4.1 Chỉ số của bộ xếp hạng trên tập kiểm định qua các epoch 26

Danh sách bảng

2.1 Các thuật toán cho SUKP được dùng để so sánh. 8

3.1 Đối chiếu khái niệm học tăng cường với tìm kiếm cục bộ. 18

3.2 Siêu tham số của DQN. 19

3.3 So sánh hai phương án học hàm xếp hạng. 21

4.1 Kết quả trên Set I 23

4.2 Kết quả trên Set II 24

4.3 Tóm tắt kết quả của bộ giải (luật tỉ số, một lần chạy 30 giây) so với BKS. 24

4.4 Hành vi tìm kiếm của bộ giải với luật tỉ số 25

4.5 Chất lượng xếp hạng trên tập kiểm định: luật tỉ số của E2LS so với bộ xếp hạng học được (checkpoint epoch 30). 26

4.6 Chi phí của 200 bước tìm kiếm trên `sukp_1000_1000_0.10_0.75` (CPU). . 27

4.7 Luật tỉ số so với các hàm xếp hạng học được trên bộ chuẩn 28

Chương 1. Tổng quan

1.1 Đặt vấn đề

Bài toán cái túi (knapsack) là một trong những mô hình kinh điển nhất của tối ưu tổ hợp: chọn một tập con các vật có giá trị sao cho tổng giá trị lớn nhất mà tổng trọng lượng không vượt quá sức chứa của túi [12]. Trong nhiều ứng dụng thực tế, chi phí của một lựa chọn không phải là tổng chi phí của từng vật một cách độc lập, vì các vật có thể *dùng chung tài nguyên*. Khi đó tài nguyên dùng chung chỉ phải trả một lần. Bài toán cái túi hợp tập (Set-Union Knapsack Problem, SUKP) do Goldschmidt, Nehme và Yu đề xuất năm 1994 [10] mô hình hóa đúng tình huống này: mỗi *item* có một lợi nhuận và là một tập các *element* có trọng số; trọng lượng của một lựa chọn là tổng trọng số của *hợp* các element mà các item được chọn phủ lên.

SUKP xuất hiện trong nhiều lĩnh vực như sản xuất linh hoạt [10], ra quyết định tài chính [12,14], nén dữ liệu, tối ưu mạng định nghĩa bằng phần mềm và lựa chọn dự án đầu tư [24,28]. Bài toán là NP-khó: khi không có hai item nào dùng chung element, SUKP trở thành bài toán cái túi 0-1 [1,10]. Vì vậy, với các instance có hàng trăm đến hàng nghìn item, các phương pháp chính xác khó áp dụng, và các thuật toán metaheuristic, đặc biệt là tìm kiếm cục bộ dựa trên tabu [24,25,28], đang cho kết quả tốt nhất.

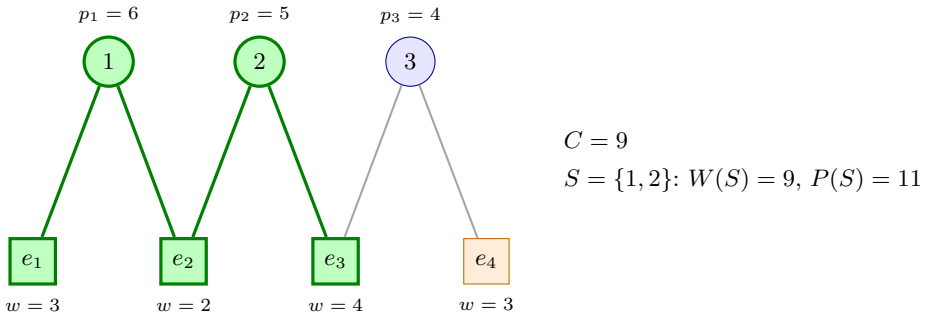
Các thuật toán tìm kiếm cục bộ tốt nhất hiện nay đều dựa vào những *luật đánh giá viết tay*. Thuật toán E2LS [28], một trong những phương pháp mạnh nhất cho SUKP, quyết định thêm item nào vào lời giải dựa trên tỉ số giữa lợi nhuận của item và phần trọng số mà item đó làm tăng thêm. Tỉ số này đơn giản, rẻ và hiệu quả, nhưng về bản chất là một đánh giá *tham lam một bước*: nó đo lợi ích tức thời của một nước đi mà không biết nước đi đó mở ra hay đóng lại những khả năng nào ở các bước sau, chẳng hạn khi hai item dùng chung nhiều element nên chọn cùng nhau thì tốt hơn chọn riêng lẻ.

Song song với đó, học máy cho tối ưu tổ hợp đã phát triển mạnh trong khoảng mười năm gần đây [3]. Một hướng đặc biệt hiệu quả là *không thay thế thuật toán tối ưu, mà chỉ học một thành phần của nó*: học quy tắc rẽ nhánh trong branch-and-bound [7], hay học hàm heuristic trong giải thuật đàn kiến [26]. Cách tiếp cận này giữ nguyên các đảm bảo và cơ chế tìm kiếm đã được kiểm chứng của thuật toán gốc, đồng thời cho phép mô hình học được những quy luật mà luật viết tay bỏ sót.

Luận văn này theo hướng tiếp cận trên cho SUKP. Câu hỏi nghiên cứu đặt ra là: *có thể học một hàm đánh giá nước đi tốt hơn tỉ số giá trị trên trọng số, và thay nó vào đúng một vị trí trong E2LS, để cải thiện chất lượng tìm kiếm hay không?*

1.2 Bài toán cái túi hợp tập

Hình 1.1 minh họa một instance nhỏ. Có ba item với lợi nhuận $p_1 = 6$, $p_2 = 5$, $p_3 = 4$ và bốn element với trọng số lần lượt là $3, 2, 4, 3$; sức chứa $C = 9$. Item 1 phủ $\{e_1, e_2\}$, item 2 phủ $\{e_2, e_3\}$, item 3 phủ $\{e_3, e_4\}$. Nếu tính chi phí theo từng item như bài toán cái túi thông thường, chọn item 1 và 2 tốn $5 + 6 = 11 > C$. Nhưng trong SUKP, element e_2 được hai item dùng chung chỉ tính một lần, nên lời giải $S = \{1, 2\}$ có trọng lượng $W(S) = 3 + 2 + 4 = 9 \leq C$ và lợi nhuận $P(S) = 11$, là lời giải tối ưu.



Hình 1.1: Một instance SUKP nhỏ dưới dạng đồ thị hai phía item–element. Item được chọn và các element chúng phủ được tô màu xanh; element e_2 dùng chung nên chỉ tính trọng số một lần.

Biểu diễn đồ thị hai phía trong Hình 1.1 không chỉ để minh họa. Nó cũng là cấu trúc dữ liệu đầu vào của các mạng nơ-ron đồ thị được dùng trong luận văn (Chương 3): giá trị của việc thêm một item phụ thuộc vào việc các element của nó đã được phủ hay chưa, và được những item nào khác dùng chung.

1.3 Mục tiêu và phạm vi nghiên cứu

Mục tiêu tổng quát của luận văn là xây dựng một bộ giải SUKP kết hợp tìm kiếm cục bộ với học máy, trong đó học máy chỉ đảm nhận một vai trò được xác định rõ, để mọi khác biệt về kết quả có thể quy cho thành phần được học. Cụ thể, luận văn đặt ra các mục tiêu sau:

1. Cài đặt lại thuật toán E2LS [28] bằng C++ một cách hiệu quả, có giao diện Python, và kiểm chứng chất lượng trên 60 instance chuẩn của SUKP.
2. Xác định một điểm cắm (plug-in point) cho học máy trong E2LS: bước xếp hạng các ứng viên tại mỗi nút của toán tử ADD*, nơi bài báo gốc dùng tỉ số giá trị trên trọng số.
3. Thiết kế một mạng nơ-ron đồ thị trên đồ thị hai phía item–element, có số tham số không phụ thuộc kích thước instance.
4. Huấn luyện hàm xếp hạng theo hai cách: (i) học tăng cường bằng DQN trên từng instance, và (ii) học bắt chước từ các nhãn tối ưu được sinh bằng tìm kiếm vét cạn trên các instance sinh ngẫu nhiên.

5. Đánh giá hàm xếp hạng học được so với luật tỉ số gốc về chất lượng xếp hạng và chi phí suy luận, và xây dựng quy trình so sánh hai hàm xếp hạng trên bộ chuẩn ở cùng số bước và cùng thời gian chạy.

Về phạm vi, luận văn chỉ xét SUKP (không xét biến thể BMCP), dùng bộ 60 instance chuẩn gồm Set I [11] và Set II [25], và chỉ thay đổi bước xếp hạng ứng viên của ADD*; các thành phần khác của E2LS (chọn item để bỏ, danh sách tabu, khởi tạo lại) được giữ nguyên. Toàn bộ thực nghiệm chạy trên một máy tính xách tay, không dùng GPU, nên ngân sách thời gian ngắn hơn đáng kể so với các công bố trước (Chương 4).

1.4 Đóng góp của luận văn

Các đóng góp chính của luận văn gồm:

- Một bản cài đặt E2LS bằng C++17 với trạng thái tăng dần (incremental), danh sách tabu bằng băm Zobrist 64 bit, và module Python (`sukp_cpp`) cho phép thay hàm xếp hạng ứng viên bằng một hàm Python bất kỳ. Với một lần chạy 30 giây, bản cài đặt đạt lời giải tốt nhất đã biết trên 17/30 instance của Set I và có khoảng cách trung bình 1.45% trên Set II.
- Mô hình hóa việc chọn nước đi của tìm kiếm cục bộ thành một quá trình quyết định Markov, với phần thưởng bằng mức cải thiện lời giải tốt nhất, sao cho mục tiêu học trùng với mục tiêu của tìm kiếm cục bộ.
- Một kiến trúc mạng nơ-ron đồ thị cho SUKP với ba loại nút (item, element, nút toàn cục) mà các đặc trưng đầu vào là các đại lượng quen thuộc của bài toán (trọng số tăng thêm, độ phủ của element), được dùng chung cho cả DQN và bộ xếp hạng.
- Một bộ xếp hạng học bắt chước được tham số hóa dưới dạng *hiệu chỉnh* của luật tỉ số gốc: khi chưa huấn luyện, nó xếp hạng giống hệt bài báo, và quá trình học chỉ thay đổi những chỗ có lợi. Trên các instance kiểm định, bộ xếp hạng nâng tỉ lệ chọn đúng ứng viên tối ưu từ khoảng 61% lên khoảng 71%.
- Một phân tích hành vi tìm kiếm và chi phí suy luận, cho thấy khởi tạo lại chiếm tới 21–44% số bước trên các instance nhỏ và việc gọi mạng tại mỗi nút ADD* làm bước tìm kiếm chậm đi khoảng 6 lần trên instance lớn nhất; từ đó đề xuất các hướng cải tiến cụ thể.

1.5 Cấu trúc luận văn

Phần còn lại của luận văn được tổ chức như sau. Chương 2 trình bày cơ sở lý thuyết: định nghĩa hình thức của SUKP, các thuật toán đã có, tìm kiếm cục bộ và tabu, học tăng cường, mạng nơ-ron đồ thị và các hướng học máy cho tối ưu tổ hợp. Chương 3 trình bày phương pháp đề xuất: bộ giải E2LS cài đặt bằng C++, điểm cắm học máy, kiến trúc

mạng đồ thị và hai phương án huấn luyện. Chương 4 trình bày thiết lập và kết quả thực nghiệm. Chương 5 tổng kết luận văn và đề xuất các hướng phát triển.

Chương 2. Cơ sở lý thuyết và các nghiên cứu liên quan

Chương này trình bày các kiến thức nền tảng mà luận văn sử dụng. Mục 2.1 định nghĩa bài toán SUKP và bộ dữ liệu chuẩn. Mục 2.2 tổng quan các thuật toán đã được đề xuất cho SUKP. Mục 2.3 trình bày tìm kiếm cục bộ và tìm kiếm tabu. Mục 2.4 và 2.5 lần lượt trình bày học tăng cường và mạng nơ-ron đồ thị. Mục 2.6 tổng quan các hướng học máy cho tối ưu tổ hợp và xác định vị trí của luận văn.

2.1 Bài toán cái túi hợp tập

2.1.1 Định nghĩa hình thức

Cho một tập $U = \{1, \ldots, n\}$ gồm n element, mỗi element j có trọng số $w_j > 0$, và một tập $V = \{1, \ldots, m\}$ gồm m item, mỗi item i có lợi nhuận $p_i > 0$ và tương ứng với một tập con element $U_i \subseteq U$. Quan hệ giữa item và element được cho bởi ma trận $R \in \{0,1\}^{m \times n}$, với $R_{ij} = 1$ khi và chỉ khi $j \in U_i$. Cho sức chứa $C > 0$. Với một tập item $S \subseteq V$, định nghĩa tổng lợi nhuận và tổng trọng số:

$$P(S) = \sum_{i \in S} p_i, \qquad W(S) = \sum_{j \in \bigcup_{i \in S} U_i} w_j. \tag{2.1}$$

Bài toán cái túi hợp tập là tìm

$$\max_{S \subseteq V} \; P(S) \quad \text{với ràng buộc} \quad W(S) \leq C. \tag{2.2}$$

Điểm khác biệt cốt lõi so với bài toán cái túi 0-1 nằm ở $W(S)$: trọng số của một element chỉ được tính *một lần* dù element đó thuộc nhiều item được chọn [10, 25]. Do đó chi phí của một item không cố định mà phụ thuộc vào những item đã có trong lời giải.

2.1.2 Mô hình quy hoạch nguyên 0-1

Đặt biến nhị phân $x_i = 1$ nếu item i được chọn và $y_j = 1$ nếu element j được phủ. SUKP có thể viết dưới dạng quy hoạch tuyến tính nguyên:

$$\max \quad \sum_{i=1}^{m} p_i x_i \tag{2.3}$$

$$\text{s.t.} \quad \sum_{j=1}^{n} w_j y_j \leq C, \tag{2.4}$$

$$x_i \leq y_j \qquad \forall i \in V,\ \forall j \in U_i, \tag{2.5}$$

$$x_i,\ y_j \in \{0, 1\}.$$

Ràng buộc (2.5) buộc mọi element của một item được chọn phải được phủ. Vì mục tiêu không phụ thuộc y và $w_j > 0$, ở lời giải tối ưu $y_j = 1$ chỉ khi cần thiết, nên (2.4) đúng bằng ràng buộc $W(S) \leq C$. Mô hình này có $m + n$ biến và $\sum_i |U_i|$ ràng buộc phủ. Wei và Hao cho thấy bộ giải CPLEX dựa trên mô hình 0-1 chỉ chứng minh được tối ưu cho 6 instance nhỏ nhất (85 đến 100 item và element) của bộ chuẩn trong giới hạn hai giờ [24], cho thấy phương pháp chính xác khó mở rộng.

2.1.3 Độ phức tạp và các bài toán liên quan

SUKP là NP-khó: khi các tập U_i đôi một rời nhau, $W(S) = \sum_{i \in S} \sum_{j \in U_i} w_j$ và SUKP trở thành bài toán cái túi 0-1 với trọng lượng item $\sum_{j \in U_i} w_j$ [10]. Khi mọi lợi nhuận và trọng số bằng 1, SUKP tương đương bài toán đồ thị con dày đặc nhất trên siêu đồ thị (densest k-subhypergraph) [1]. SUKP cũng có quan hệ đối ngẫu với bài toán phủ cực đại có ngân sách (BMCP) [14]: BMCP cực đại hóa tổng trọng số element được phủ với ràng buộc tổng chi phí item, tức là đổi vai trò của hàm mục tiêu và ràng buộc [28]. Arulselvan [1] chỉ ra rằng thuật toán tham lam của Khuller, Moss và Naor cho BMCP cũng cho một xấp xỉ hằng số với trường hợp đặc biệt của SUKP khi mỗi element thuộc không quá một số hằng tập.

2.1.4 Bộ dữ liệu chuẩn

Các nghiên cứu về SUKP thường đánh giá trên hai bộ dữ liệu, tổng cộng 60 instance:

- **Set I** gồm 30 instance với 85 đến 500 item và element, do He và cộng sự đề xuất [11]. Các instance chia làm ba loại theo quan hệ giữa m và n: $m > n$, $m = n$ và $m < n$.
- **Set II** gồm 30 instance lớn hơn với 585 đến 1000 item và element, do Wei và Hao đề xuất [25]. Lợi nhuận và trọng số được sinh ngẫu nhiên trong đoạn $[1, 500]$ theo cách của He và cộng sự.

Mỗi instance được đặc trưng bởi bốn tham số và được đặt tên theo dạng `sukp_`m`_`n`_`α`_`β, trong đó $\alpha = \sum_{i,j} R_{ij}/(mn)$ là mật độ của ma trận quan hệ và $\beta = C/\sum_j w_j$ là tỉ lệ sức

chứa so với tổng trọng số element [28]. Hai cặp tham số được dùng là $(\alpha, \beta) = (0.10, 0.75)$ và $(0.15, 0.85)$. Ví dụ `sukp_85_100_0.10_0.75` có 85 item, 100 element, mật độ 0.10 và sức chứa bằng 75% tổng trọng số element.

2.2 Các thuật toán giải SUKP

2.2.1 Thuật toán chính xác và xấp xỉ

Goldschmidt và cộng sự đề xuất bài toán cùng với một thuật toán quy hoạch động có thời gian chạy hàm mũ [10]. Như đã nêu ở trên, giải mô hình quy hoạch nguyên bằng bộ giải thương mại chỉ khả thi với các instance rất nhỏ [24]. Các thuật toán xấp xỉ [1] có đảm bảo lý thuyết nhưng chất lượng lời giải trên thực tế thấp hơn nhiều so với các heuristic [28]. Vì vậy hướng nghiên cứu chính cho SUKP là các thuật toán heuristic và metaheuristic.

2.2.2 Thuật toán dựa trên quần thể

He và cộng sự [11] đề xuất thuật toán bầy ong nhân tạo nhị phân (BABC), cùng với bộ dữ liệu Set I. Sau đó nhiều thuật toán bầy đàn và tiến hóa khác được đề xuất. Ozsoydan và Gölcük [18] dùng Q-learning như một hệ thống khuyến nghị chọn giữa các bộ tối ưu PSO, GA và một biến thể lai di truyền–bầy hạt (gbPSO): bộ tối ưu nào đem lại cải thiện sẽ được thưởng và được gọi thường xuyên hơn. Đây là một dạng siêu heuristic (hyper-heuristic [5]) dùng học tăng cường ở mức chọn thuật toán, khác với luận văn này, nơi học máy được dùng ở mức chọn nước đi. Gần đây, Zhao và Hifi [27] đề xuất thuật toán tiến hóa thích nghi AES-BM với tập lưu trữ các lời giải ưu tú và đa dạng. Nhìn chung, các phương pháp quần thể phức tạp hơn để thiết kế và kém hơn các phương pháp tìm kiếm cục bộ tốt nhất trên các instance lớn [28].

2.2.3 Thuật toán tìm kiếm cục bộ

Wei và Hao [24] đề xuất I2PLS, thuật toán tìm kiếm cục bộ đầu tiên cho SUKP, xen kẽ một pha tìm kiếm tabu với một pha nhiễu loạn để đến vùng chưa khám phá. KBTS [25] dùng một bộ đếm tần suất trong quá trình tìm kiếm tabu để xây dựng một lời giải "nhân" (kernel) gồm các item thường xuất hiện trong các lời giải tốt, rồi tìm kiếm tăng cường quanh nhân đó trước khi nhiễu loạn; công trình này cũng đề xuất bộ dữ liệu Set II. Các thuật toán này tìm láng giềng tốt nhất của lời giải hiện tại ở mỗi bước, nhưng láng giềng chứa nhiều nước đi chất lượng thấp, làm giảm hiệu quả [28].

Zhu và Luo [28] đề xuất E2LS (Efficient and Effective Local Search) cho cả SUKP và BMCP. Ý tưởng chính của E2LS là *thu hẹp vùng tìm kiếm* bằng tỉ số giá trị trên trọng số: chỉ thử bỏ các item có tỉ số thấp nhất, và chỉ rẽ nhánh trên các ứng viên có tỉ số cao nhất khi thêm lại. Toán tử ADD* của E2LS thêm lại nhiều item theo một cây rẽ nhánh

có giới hạn, cho phép xét nhiều tổ hợp item mà vẫn nhanh. E2LS dùng danh sách tabu dựa trên lời giải và khởi tạo lại bằng một thủ tục tham lam ngẫu nhiên. Theo báo cáo của tác giả, E2LS đạt hoặc vượt lời giải tốt nhất đã biết trên các bộ dữ liệu chuẩn. E2LS là thuật toán nền của luận văn và được trình bày chi tiết ở Mục 3.2.

Bảng 2.1 tóm tắt các thuật toán được dùng để so sánh trong Chương 4.

Bảng 2.1: Các thuật toán cho SUKP được dùng để so sánh.

Thuật toán	**Loại**	**Công bố**	**Bộ dữ liệu báo cáo**
BABC [11]	Bầy ong nhân tạo nhị phân	2018	Set I
I2PLS [24]	Tìm kiếm cục bộ hai pha lặp	2019	Set I
KBTS [25]	Tìm kiếm tabu dựa trên nhân	2021	Set I, Set II
E2LS [28]	Tìm kiếm cục bộ với ADD*	2022	Set I, Set II
QL, gbPSO [18]	Q-learning chọn bộ tối ưu; PSO lai GA	2023	Set I
AES-BM [27]	Tiến hóa thích nghi	2024	Set I

2.3 Tìm kiếm cục bộ và tìm kiếm tabu

Tìm kiếm cục bộ bắt đầu từ một lời giải và lặp lại việc di chuyển sang một lời giải *láng giềng*, tức là một lời giải nhận được bằng một thay đổi nhỏ như thêm, bỏ hoặc hoán đổi một phần tử. Một quy tắc chọn nước đi (move evaluation) quyết định láng giềng nào được chọn. Nếu chỉ chấp nhận nước đi cải thiện, thuật toán sẽ dừng ở một cực trị địa phương.

Tìm kiếm tabu [9] cho phép đi sang lời giải tệ hơn để thoát khỏi cực trị địa phương, và dùng một bộ nhớ ngắn hạn (danh sách tabu) để cấm quay lại các trạng thái hoặc nước đi gần đây, tránh đi vòng. Có hai dạng chính: tabu theo *thuộc tính* cấm đảo ngược một nước đi trong một số bước, và tabu theo *lời giải* cấm thăm lại chính các lời giải đã thăm. Tabu theo lời giải thường được cài đặt bằng hàm băm: E2LS dùng ba vector băm có độ dài 10^8, và một lời giải bị cấm nếu cả ba vị trí băm của nó đều đã được đánh dấu [28]. Hai khái niệm quan trọng khác của metaheuristic là *tăng cường* (intensification), tức tập trung tìm quanh các vùng tốt, và *đa dạng hóa* (diversification), tức đưa tìm kiếm đến vùng mới, chẳng hạn bằng khởi tạo lại.

Hiệu quả của tìm kiếm cục bộ phụ thuộc lớn vào quy tắc chọn nước đi. Quy tắc này thường là một heuristic viết tay, như tỉ số giá trị trên trọng số trong E2LS. Luận văn giữ nguyên khung tìm kiếm và chỉ học lại quy tắc này.

2.4 Học tăng cường

2.4.1 Quá trình quyết định Markov

Học tăng cường [20] nghiên cứu cách một tác tử (agent) học chọn hành động thông qua tương tác với môi trường. Bài toán được hình thức hóa bằng quá trình quyết định Markov (MDP) gồm tập trạng thái $\mathcal{S}$, tập hành động $\mathcal{A}(s)$ khả dụng tại trạng thái s, hàm chuyển trạng thái và hàm phần thưởng. Tại bước t, tác tử ở trạng thái s_t, chọn hành động a_t theo chính sách $\pi(a \mid s)$, nhận phần thưởng r_{t+1} và chuyển sang s_{t+1}. Mục tiêu là cực đại hóa *return* kỳ vọng

$$G_t = \sum_{k \geq 0} \gamma^k r_{t+k+1}, \tag{2.6}$$

với hệ số chiết khấu $\gamma \in [0, 1)$. Giá trị $1/(1-\gamma)$ có thể hiểu là tầm nhìn hiệu dụng tính theo số bước.

Hàm giá trị hành động $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$ đo mức tốt của việc chọn a tại s rồi đi theo π. Hàm giá trị tối ưu Q^* thỏa phương trình Bellman

$$Q^*(s, a) = \mathbb{E}\Big[\, r_{t+1} + \gamma \max_{a' \in \mathcal{A}(s_{t+1})} Q^*(s_{t+1}, a') \;\Big|\; s_t = s,\ a_t = a \Big], \tag{2.7}$$

và chính sách tối ưu chọn $\arg\max_a Q^*(s, a)$. Đối với người làm tối ưu, $Q^*(s, a)$ chính là *giá trị còn lại* (cost-to-go) của nước đi a: tổng cải thiện tốt nhất có thể đạt được nếu đi a rồi tiếp tục tối ưu; phương trình (2.7) là đệ quy của quy hoạch động [4].

2.4.2 Q-learning

Khi không gian trạng thái quá lớn để giải (2.7) trực tiếp, Q-learning [23] ước lượng Q^* từ các chuyển tiếp mẫu (s, a, r, s'):

$$Q(s, a) \leftarrow Q(s, a) + \alpha \Big[\, r + \gamma \max_{a'} Q(s', a') - Q(s, a) \,\Big], \tag{2.8}$$

với bước học α. Q-learning dạng bảng hội tụ về Q^* khi mọi cặp trạng thái–hành động được thăm vô hạn lần và bước học giảm dần phù hợp [23]. Để mọi hành động đều được thử, tác tử cần *thăm dò*: chính sách ε-greedy chọn hành động ngẫu nhiên với xác suất ε; chính sách softmax (Boltzmann) chọn a với xác suất tỉ lệ với $\exp(Q(s, a)/\tau)$, với nhiệt độ τ. Thăm dò trong học tăng cường tương ứng với đa dạng hóa trong metaheuristic.

2.4.3 Mạng Q sâu

Với SUKP, mỗi lời giải là một vector nhị phân độ dài m, nên có tới 2^m trạng thái và không thể lưu Q dưới dạng bảng. Mạng Q sâu (DQN) [17] xấp xỉ Q bằng một mạng nơ-ron Q_θ và cực tiểu hóa sai số so với đích Bellman:

$$y = r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a'), \qquad \mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}}\Big[\big(Q_\theta(s, a) - y\big)^2\Big]. \tag{2.9}$$

Hai kỹ thuật ổn định quá trình học là: (i) *bộ nhớ phát lại* (replay buffer) $\mathcal{D}$ lưu các chuyển tiếp cũ và lấy mẫu ngẫu nhiên, làm giảm tương quan giữa các mẫu liên tiếp; và (ii) *mạng đích* $Q_{\bar{\theta}}$ là bản sao của Q_θ được cập nhật định kỳ, tránh việc đích học thay đổi liên tục. Khác với Q-learning dạng bảng, DQN không có đảm bảo hội tụ, nên hiệu quả của nó được đánh giá bằng thực nghiệm.

2.5 Mạng nơ-ron đồ thị

Mạng nơ-ron đồ thị (GNN) học biểu diễn cho các nút của một đồ thị bằng cơ chế *truyền thông điệp* (message passing) [8]. Mỗi nút v có một vector ẩn h_v; ở mỗi vòng, nút tổng hợp thông điệp từ các nút kề rồi cập nhật:

$$h_v^{(k+1)} = \phi\Big(h_v^{(k)},\ \operatorname*{AGG}_{u\in\mathcal{N}(v)} \psi\big(h_u^{(k)}\big)\Big), \tag{2.10}$$

trong đó ϕ, ψ là các mạng nơ-ron (thường là MLP) và AGG là một phép tổng hợp bất biến với hoán vị như tổng hoặc trung bình. Sau K vòng, h_v chứa thông tin từ lân cận bán kính K của v. Hai tính chất của GNN đặc biệt phù hợp với tối ưu tổ hợp. Thứ nhất, kết quả *bất biến với cách đánh số* item và element. Thứ hai, các hàm ϕ, ψ được dùng chung cho mọi nút, nên *số tham số không phụ thuộc kích thước đồ thị*, và một mô hình có thể áp dụng cho instance với số item và element bất kỳ.

Nhiều bài toán tối ưu có cấu trúc hai phía tự nhiên. Gasse và cộng sự [7] biểu diễn một bài toán quy hoạch nguyên hỗn hợp bằng đồ thị hai phía biến–ràng buộc để học quy tắc rẽ nhánh. SUKP cũng có cấu trúc hai phía item–element (Hình 1.1), nên luận văn dùng một GNN trên đồ thị này. Chuẩn hóa tầng (layer normalization) [2] và kết nối tắt (residual) được dùng để ổn định quá trình học khi xếp chồng nhiều vòng truyền thông điệp.

2.6 Học máy cho tối ưu tổ hợp

Bengio, Lodi và Prouvost [3] phân loại các cách kết hợp học máy với tối ưu tổ hợp thành ba nhóm: (i) học máy giải bài toán *từ đầu đến cuối* (end-to-end); (ii) học máy cung cấp thông tin cho thuật toán tối ưu trước khi chạy; và (iii) học máy được gọi *bên trong* thuật toán tối ưu để đưa ra quyết định lặp lại.

Thuộc nhóm thứ nhất, mạng con trỏ (Pointer Networks) [22] và mô hình attention của Kool và cộng sự [16] xây dựng lời giải bài toán định tuyến từng bước. Khalil và cộng sự [13] kết hợp nhúng đồ thị với Q-learning để xây dựng lời giải cho các bài toán trên đồ thị. Các phương pháp này nhanh khi suy luận nhưng thường kém các bộ giải chuyên biệt về chất lượng lời giải trên instance lớn.

Thuộc nhóm thứ ba, Gasse và cộng sự [7] học bắt chước quy tắc rẽ nhánh mạnh (strong branching) trong branch-and-bound bằng GNN trên đồ thị hai phía, vẫn giữ tính chính xác của thuật toán gốc. DeepACO [26] giữ nguyên giải thuật đàn kiến và chỉ học

hàm heuristic η dùng trong xác suất xây dựng lời giải, huấn luyện bằng gradient chính sách trên nhiều instance. DeepACO cải thiện các biến thể đàn kiến trên tám bài toán, trong đó có bài toán cái túi nhiều ràng buộc. Các siêu heuristic [5], trong đó có cách tiếp cận Q-learning cho SUKP của Ozsoydan và Gölcük [18], cũng thuộc họ này nhưng học ở mức chọn thuật toán con thay vì chọn nước đi.

Luận văn thuộc nhóm thứ ba và gần với tinh thần của Gasse và cộng sự cũng như DeepACO: khung tìm kiếm cục bộ E2LS được giữ nguyên, và học máy chỉ thay *một* heuristic viết tay là bước xếp hạng ứng viên của ADD*. So với DeepACO, điểm khác biệt là hàm xếp hạng của luận văn phụ thuộc vào lời giải hiện tại và được tính lại tại mỗi nút tìm kiếm, thay vì được tính một lần cho mỗi instance. Điều này cho phép biểu diễn những quan hệ phụ thuộc trạng thái như trọng số tăng thêm $AW(S, i)$, nhưng có chi phí suy luận lớn hơn. Đánh đổi này được phân tích trong Chương 4.

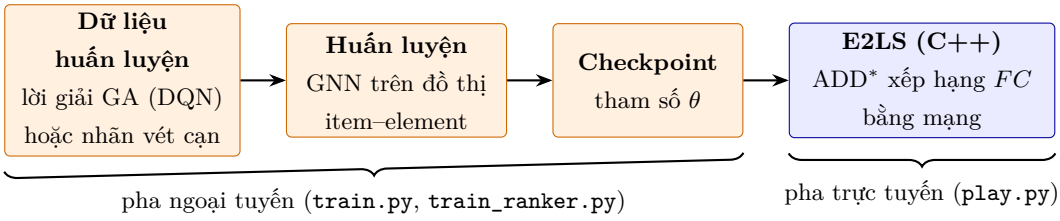
Chương 3. Phương pháp đề xuất

Chương này trình bày phương pháp của luận văn. Mục 3.1 mô tả kiến trúc tổng thể. Mục 3.2 trình bày thuật toán E2LS và bản cài đặt bằng C++. Mục 3.3 xác định điểm cắm học máy. Mục 3.4 trình bày mạng nơ-ron đồ thị dùng chung. Mục 3.5 và 3.6 trình bày hai phương án huấn luyện: DQN theo từng instance và bộ xếp hạng học bắt chước. Mục 3.7 so sánh hai phương án.

3.1 Kiến trúc tổng thể

Nguyên tắc thiết kế của luận văn là *chỉ thay đúng một thành phần* của một thuật toán tìm kiếm cục bộ mạnh. Toàn bộ khung E2LS được giữ nguyên: cách chọn item để bỏ, danh sách tabu, quy tắc chấp nhận lời giải và cơ chế khởi tạo lại. Mạng nơ-ron chỉ làm một việc: chấm điểm các ứng viên tại mỗi nút của toán tử ADD*. Nhờ đó, mọi khác biệt về kết quả giữa bộ giải gốc và bộ giải có học máy đều có thể quy cho hàm xếp hạng.

Hình 3.1 minh họa quy trình hai pha. Ở pha ngoại tuyến (offline), một mạng nơ-ron đồ thị được huấn luyện và lưu thành tệp checkpoint. Ở pha trực tuyến, bộ giải C++ chạy E2LS; mỗi khi ADD* cần xếp hạng ứng viên, nó gọi một hàm Python, hàm này đưa lời giải hiện tại qua mạng và trả về điểm số.



Hình 3.1: Quy trình hai pha: huấn luyện mạng ngoại tuyến, sau đó dùng mạng để xếp hạng ứng viên trong ADD*.

3.2 Thuật toán E2LS và bản cài đặt C++

3.2.1 Trọng số tăng thêm và tỉ số giá trị trên trọng số

Gọi $c_j(S) = |\{i \in S : j \in U_i\}|$ là số item trong S phủ element j. E2LS dựa trên hai khái niệm [28]. *Trọng số tăng thêm* của item i đối với lời giải S là

$$AW(S,i) = \begin{cases} W(S \cup \{i\}) - W(S) = \sum_{j \in U_i,\ c_j(S)=0} w_j, & i \notin S, \\ W(S) - W(S \setminus \{i\}) = \sum_{j \in U_i,\ c_j(S)=1} w_j, & i \in S, \end{cases} \tag{3.1}$$

tức là trọng số mới bị phủ khi thêm i, hoặc trọng số được giải phóng khi bỏ i (các element chỉ có i phủ). *Tỉ số giá trị trên trọng số* là

$$R(S,i) = \frac{p_i}{AW(S,i)}. \tag{3.2}$$

Item có R lớn nên được thêm vào hoặc giữ lại; item có R nhỏ nên bị bỏ ra trước. Một item $i \notin S$ có $AW(S,i) = 0$ được gọi là item *miễn phí*: thêm nó làm tăng lợi nhuận mà không tăng trọng số, nên luôn được thêm. Lưu ý rằng $R(S,i)$ phụ thuộc vào S: cùng một item có thể đắt khi đứng một mình nhưng gần như miễn phí khi các element của nó đã được những item khác phủ.

3.2.2 Khởi tạo tham lam ngẫu nhiên

Thủ tục Random_Greedy(t) (Thuật toán 1) xây dựng một lời giải đầy đủ: lặp lại việc thêm mọi item miễn phí, rồi lấy mẫu có hoàn lại t ứng viên khả thi và thêm ứng viên có tỉ số R lớn nhất, cho đến khi không còn item nào vừa túi. Tham số $t = \sqrt{\max(m,n)}$ cân bằng giữa chất lượng và độ đa dạng của lời giải khởi tạo [28]. Thủ tục này cũng được dùng để khởi tạo lại khi tìm kiếm bị kẹt.

Thuật toán 1 Random_Greedy(t)

Đầu vào: số lần lấy mẫu t

Đầu ra: lời giải khả thi S

$S \leftarrow \emptyset$
loop
  thêm vào S mọi item $i \notin S$ có $AW(S,i) = 0$ (lặp đến điểm bất động)
  $FC \leftarrow \{i \notin S : AW(S,i) > 0,\ W(S) + AW(S,i) \le C\}$
  if $FC = \emptyset$ **then return** S
  end if
  lấy mẫu có hoàn lại t item từ FC; gọi $i^\star$ là item có $R(S,i)$ lớn nhất
  $S \leftarrow S \cup \{i^\star\}$
end loop

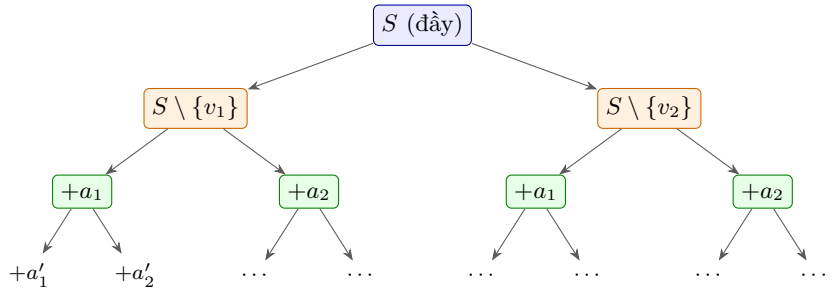
3.2.3 Bước tìm kiếm cục bộ và toán tử ADD*

Mỗi bước tìm kiếm cục bộ (Thuật toán 2) thử bỏ lần lượt từng item trong r_{num} item có tỉ số R nhỏ nhất của lời giải hiện tại S. Sau mỗi lần bỏ, toán tử ADD* thêm lại item theo một cây rẽ nhánh: tại mỗi nút, ADD* thêm mọi item miễn phí (bỏ qua những lời giải nằm trong danh sách tabu), tính tập ứng viên khả thi FC, *xếp hạng* FC, rồi rẽ nhánh trên a_{num} ứng viên đứng đầu (bỏ qua ứng viên dẫn tới lời giải tabu) và đệ quy cho tới khi không còn item nào vừa túi. Mọi lời giải không tabu gặp trên cây đều được so với lời giải tốt nhất của bước, S_b. Cuối bước, thuật toán di chuyển tới S_b *kể cả khi* S_b tệ hơn S, rồi đưa S_b vào danh sách tabu. Nếu mọi lời giải láng giềng đều tabu, thuật toán khởi tạo lại bằng RANDOM_GREEDY. Với tham số mặc định $r_{num} = a_{num} = 2$, mỗi bước khám phá hai cây nhị phân như Hình 3.2.

Thuật toán 2 Một bước tìm kiếm cục bộ của E2LS và toán tử ADD*

```
function LOCAL_SEARCH(S)
    S_b ← ∅
    for mỗi v trong r_num item của S có R(S, v) nhỏ nhất do
        ADDSTAR(S \ {v}, S_b)
    end for
    return S_b                                  ▷ ∅ nếu mọi lời giải gặp được đều tabu
end function

function ADDSTAR(S, S_b)
    thêm vào S mọi item miễn phí mà lời giải thu được không tabu; cập nhật S_b
    FC ← {i ∉ S : AW(S, i) > 0,  W(S) + AW(S, i) ≤ C}
    L ← xếp hạng(FC)                  ▷ bài báo: giảm dần theo R(S, i); luận văn: theo mạng
    for mỗi i trong a_num phần tử đầu của L do
        if S ∪ {i} không tabu then
            cập nhật S_b bằng S ∪ {i}; ADDSTAR(S ∪ {i}, S_b)
        end if
    end for
end function
```



Hình 3.2: Cây tìm kiếm của một bước E2LS với $r_{num} = a_{num} = 2$. v_1, v_2: hai item có tỉ số nhỏ nhất bị thử bỏ; a_1, a_2: hai ứng viên được xếp hạng cao nhất tại mỗi nút ADD*. Một nhánh kết thúc khi không còn item nào vừa túi.

Toán tử ADD* có vùng tìm kiếm rộng: với $a_{num} = m$, nó duyệt mọi tổ hợp item có thể thêm và trở thành một thuật toán chính xác cho bài toán con khi các item hiện có được cố định [28]. Việc giới hạn $a_{num} = 2$ đánh đổi độ đầy đủ lấy tốc độ, và *thứ tự xếp hạng quyết định nhánh nào được khám phá*. Đây chính là lý do bước xếp hạng là điểm cắm học máy (Mục 3.3).

3.2.4 Cài đặt bằng C++ và giao diện Python

Bộ giải được cài đặt bằng C++17 (biên dịch với `-O3`) và đóng gói thành module Python `sukp_cpp` bằng pybind11. Các điểm chính của bản cài đặt như sau.

Trạng thái tăng dần. Bộ giải lưu $c_j(S)$ cho mọi element. Thêm hoặc bỏ item i chỉ cập nhật các element trong U_i, với chi phí $O(|U_i|)$; $W(S)$ chỉ thay đổi khi một c_j chuyển giữa 0 và 1. Nhờ đó $AW(S, i)$ được tính trong $O(|U_i|)$ mà không cần tính lại $W(S)$ từ đầu.

Danh sách tabu bằng băm Zobrist. Thay cho ba vector băm của bài báo gốc, mỗi item được gán một số ngẫu nhiên 64 bit, và giá trị băm của lời giải là phép XOR các số của những item được chọn [29]. Giá trị này được cập nhật trong $O(1)$ khi thêm hoặc bỏ item, và giá trị băm của $S \cup \{i\}$ được tính trước khi thực sự thêm i. Danh sách tabu là một bảng băm chứa mọi lời giải đã được chấp nhận. Với 64 bit, xác suất hai lời giải khác nhau trùng giá trị băm là không đáng kể, nên danh sách tabu gần như chính xác.

Một điểm diễn giải. Mã giả của bài báo gốc kiểm tra tabu trên $S \setminus \{v\}$ trước khi gọi ADD*, nhưng điều kiện này mâu thuẫn với phần mô tả bằng lời. Bản cài đặt theo phần mô tả: ADD* luôn được gọi và chỉ những lời giải mà nó gặp mới được xét điều kiện tabu.

Giao diện hàm xếp hạng. Hàm `sukp_cpp.solve(instance, params, scorer)` nhận thêm một đối số tùy chọn `scorer`. Nếu có, tại mỗi nút ADD* có *nhiều hơn* a_{num} ứng viên, bộ giải gọi `scorer(selected, candidates)` với vector chọn hiện tại và danh sách ứng viên, nhận lại một điểm cho mỗi ứng viên và sắp xếp giảm dần. Khi $|FC| \leq a_{num}$, mọi ứng viên đều được rẽ nhánh nên thứ tự không ảnh hưởng tới tập lời giải được khám phá, và bộ giải không gọi mạng. Cùng một mã nguồn chạy được cả với luật tỉ số gốc và với mạng, nên so sánh giữa hai cách là công bằng về mặt cài đặt. Mọi lời giải trả về được kiểm tra lại bằng một bản cài đặt Python độc lập.

3.3 Điểm cắm học máy: xếp hạng ứng viên của ADD*

Hình thức hóa, một *hàm xếp hạng* là một hàm $\sigma(S, i) \in \mathbb{R}$ cho mỗi lời giải bộ phận S tại một nút ADD* và mỗi ứng viên $i \in FC(S)$; ADD* rẽ nhánh trên a_{num} ứng viên có σ lớn nhất. Bài báo gốc dùng $\sigma(S, i) = R(S, i)$. Đây là một đánh giá *tham lam một bước*: nó đo lợi nhuận trên mỗi đơn vị trọng số tăng thêm ngay lập tức, mà không xét việc thêm i ảnh hưởng thế nào tới các lựa chọn sau. Ví dụ, hai item có tỉ số trung bình nhưng dùng chung nhiều element có thể cùng nhau tốt hơn một item có tỉ số cao nhưng

phủ các element riêng biệt.

Một hàm xếp hạng lý tưởng nên ước lượng *giá trị tốt nhất đạt được sau khi thêm i rồi tiếp tục thêm item một cách tối ưu*, tức là một giá trị còn lại (cost-to-go) theo nghĩa quy hoạch động. Luận văn học hàm này theo hai cách. Cách thứ nhất dùng học tăng cường: $\sigma(S, i) = Q_\theta(s, \text{add } i)$ (Mục 3.5). Cách thứ hai dùng học có giám sát từ các nhãn tối ưu được tính bằng vét cạn (Mục 3.6). Cả hai đều dùng chung một kiến trúc mạng đồ thị (Mục 3.4).

3.4 Mạng nơ-ron đồ thị trên đồ thị hai phía item–element

3.4.1 Biểu diễn đồ thị

Một instance được biểu diễn bằng đồ thị hai phía có m nút item, n nút element và các cạnh cho bởi ma trận quan hệ R, cùng với một nút toàn cục g nối với mọi nút. Ma trận R cố định cho mỗi instance, còn đặc trưng của các nút được tính lại từ vector chọn x_S ở mỗi lần gọi mạng. Các đặc trưng đầu vào đều là những đại lượng quen thuộc của SUKP, được chuẩn hóa để không phụ thuộc thang đo của instance. Gọi $\bar{p}$ và $\bar{w}$ là lợi nhuận và trọng số trung bình, và $u = \frac{1}{m}\sum_i \sum_{j \in U_i} w_j$ là đơn vị trọng số (tổng trọng số trung bình của các element thuộc một item).

- **Item i:** đã được chọn hay chưa (x_i); lợi nhuận chuẩn hóa $p_i/\bar{p}$; trọng số tăng thêm khi thêm AW_i^+/u và khi bỏ AW_i^-/u theo (3.1); còn vừa túi hay không; và một độ hiệu quả $\log(p_i/\bar{p}) - \log\Big((AW_i^+ + AW_i^-)/u + 0.1\Big)$. Bộ xếp hạng (Mục 3.6) có thêm đặc trưng "là ứng viên" ($i \in FC$).
- **Element j:** trọng số chuẩn hóa $w_j/\bar{w}$; đã được phủ hay chưa ($c_j > 0$); chỉ được phủ đúng một lần hay không ($c_j = 1$); và tỉ lệ $c_j/\deg(j)$, với $\deg(j)$ là số item chứa j.
- **Nút toàn cục:** mức sử dụng sức chứa $W(S)/C$; sức chứa còn lại $(C - W(S))/u$; tỉ lệ item được chọn $|S|/m$; và với DQN thêm khoảng cách tới lời giải tốt nhất (Mục 3.5).

Mỗi loại đặc trưng được nhúng thành vector $d = 64$ chiều bằng một MLP hai tầng riêng.

3.4.2 Truyền thông điệp

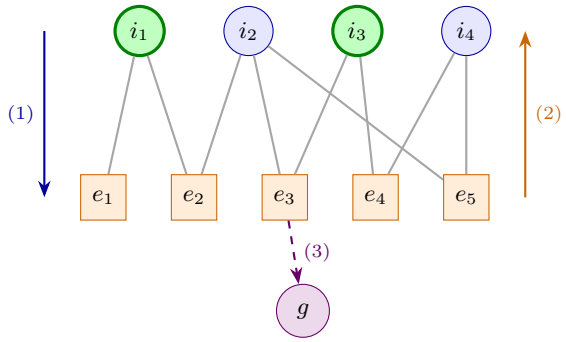
Mạng gồm $K = 3$ vòng truyền thông điệp. Ở mỗi vòng, thông tin đi từ item sang element, từ element sang item, rồi tổng hợp lên nút toàn cục (Hình 3.3):

$$h_j^E \leftarrow \mathrm{LN}\Big(h_j^E + \phi_E\Big[\, h_j^E,\ \operatorname*{mean}_{i:\, j \in U_i} h_i^I,\ \operatorname*{mean}_{i \in S:\, j \in U_i} h_i^I \,\Big]\Big), \tag{3.3}$$

$$h_i^I \leftarrow \mathrm{LN}\Big(h_i^I + \phi_I\Big[\, h_i^I,\ \operatorname*{mean}_{j \in U_i} h_j^E,\ g \,\Big]\Big), \tag{3.4}$$

$$g \leftarrow \mathrm{LN}\Big(g + \phi_G\Big[\, g,\ \operatorname*{mean}_{i} h_i^I,\ \operatorname*{mean}_{j} h_j^E \,\Big]\Big), \tag{3.5}$$

trong đó $[\cdot]$ là phép nối vector, ϕ_E, ϕ_I, ϕ_G là các MLP hai tầng, và LN là chuẩn hóa tầng [2]. Trong (3.3), mỗi element đọc riêng trung bình của *mọi* item chứa nó và trung bình của các item *đã chọn* chứa nó, nên mạng phân biệt được element đã bị phủ bởi những item nào. Có thể hình dung mạng là một phép "tính AW mở rộng" được học: thay vì chỉ hỏi element đã được phủ hay chưa, mạng tổng hợp thông tin từ những item cùng chia sẻ element.



Hình 3.3: Một vòng truyền thông điệp trên đồ thị hai phía: (1) item gửi tới element, (2) element gửi tới item, (3) tổng hợp lên nút toàn cục g. Item màu xanh lá là item đã được chọn.

Đầu ra được tính bởi các MLP *dùng chung cho mọi item*, nhận đầu vào $[h_i^I, g]$. Vì mọi tham số đều dùng chung giữa các nút, số tham số không phụ thuộc m hay n, và cùng một checkpoint có thể chạy trên instance với kích thước bất kỳ. Ma trận kề được lưu dưới dạng dày, nên mỗi vòng truyền thông điệp tốn $O(mnd)$ phép tính; với $m = n = 1000$ đây là chi phí chính của mỗi lần gọi mạng.

3.5 Phương án 1: DQN huấn luyện trên từng instance

3.5.1 Mô hình hóa quá trình quyết định Markov

Phương án thứ nhất mô hình hóa tìm kiếm cục bộ như một MDP và học Q_θ bằng DQN [17]. Bảng 3.1 đối chiếu các khái niệm học tăng cường với ngôn ngữ của tìm kiếm cục bộ.

Bảng 3.1: Đối chiếu khái niệm học tăng cường với tìm kiếm cục bộ.

Học tăng cường	**Tìm kiếm cục bộ**	**Trong luận văn**
Trạng thái s	Lời giải hiện tại	(x_S, g)
Hành động a	Một nước đi	Thêm hoặc bỏ một item
Phần thưởng r	Mức cải thiện sau một nước	Mức cải thiện lời giải tốt nhất
Episode	Một lượt tìm kiếm từ một lời giải đầu	100 nước từ một lời giải GA
$Q(s, a)$	Giá trị còn lại của nước đi	Điểm xếp hạng ứng viên của ADD*
Thăm dò	Đa dạng hóa	ε-greedy, softmax
γ	Trọng số cho cải thiện ở xa	0.99, tầm nhìn khoảng 100 bước
Che hành động	Chỉ xét nước khả thi	Che nước vượt sức chứa

Trạng thái. $s = (x_S, g)$, với $x_S \in \{0,1\}^m$ là vector chọn và

$$g = \frac{P_{best} - P(S)}{\bar{p}} \tag{3.6}$$

là khoảng cách tới lời giải tốt nhất của episode. Thành phần g là cần thiết vì phần thưởng phụ thuộc vào P_{best}: nếu thiếu nó, cùng một vector x_S có thể có giá trị khác nhau và trạng thái không còn tính Markov.

Hành động. Có $2m$ hành động: thêm item i (nếu $i \notin S$ và còn vừa túi) hoặc bỏ item i (nếu $i \in S$). Các hành động không khả thi bị che. Sau mỗi nước đi thành công, nước đi ngược lại bị cấm trong 30 bước, tức là một danh sách tabu theo thuộc tính. Mạng có thêm một đầu ra "dừng" nhưng đầu ra này luôn bị che khi chọn hành động.

Phần thưởng. Chỉ những lời giải tốt nhất mới được thưởng:

$$r_t = \frac{\max\big(0,\ P(S_{t+1}) - P_{best}\big)}{\bar{p}}, \qquad \text{sau đó} \quad P_{best} \leftarrow \max\big(P_{best}, P(S_{t+1})\big). \tag{3.7}$$

Tổng phần thưởng của một episode có dạng tổng lồng nhau (telescoping) và bằng đúng $(P_{best}^{\text{cuối}} - P_{best}^{\text{đầu}})/\bar{p}$, tức mức cải thiện lời giải tốt nhất. Như vậy *mục tiêu học trùng với mục tiêu của tìm kiếm cục bộ*. Việc bỏ item không bị phạt, nên tác tử được phép đi qua lời giải tệ hơn như tìm kiếm tabu. Tác tử cũng không thể kiếm thưởng bằng cách đi vòng: bỏ rồi thêm lại cùng một item không tạo ra lời giải tốt nhất mới.

3.5.2 Lời giải khởi đầu bằng giải thuật di truyền

Mỗi episode bắt đầu từ một lời giải khác nhau do một giải thuật di truyền (GA) nhị phân sinh ra, dùng thư viện MEALPY [21]: quần thể 50, 100 thế hệ, chọn lọc tournament kích thước 10, lai ghép đồng nhất với xác suất 0.9 và đột biến lật bit với xác suất 0.4. Một cá thể được giải mã bằng cách thêm lần lượt các item có bit 1 và bỏ qua item không còn vừa túi, nên mọi cá thể đều khả thi. Lý do dùng GA là *phân phối huấn luyện phải khớp với phân phối sử dụng*: tìm kiếm cục bộ thực tế luôn làm việc quanh các lời giải gần đầy và chất lượng cao, nên nếu tác tử chỉ học từ lời giải ngẫu nhiên, hàm Q sẽ không

chính xác ở đúng vùng mà nó được dùng. Mỗi instance có 1000 lần chạy GA độc lập, cho 1000 lời giải khởi đầu vừa tốt vừa đa dạng.

3.5.3 Huấn luyện

Mạng Q_θ có hai đầu ra cho mỗi item, $Q(s, \text{add } i) = \psi_{add}[h_i^I, g]$ và $Q(s, \text{remove } i) = \psi_{rem}[h_i^I, g]$, được huấn luyện bằng hàm mất mát (2.9). Khi tính đích Bellman, phép lấy max chỉ chạy trên các hành động khả thi ở trạng thái kế tiếp, nên các đầu ra không khả thi không bao giờ được dùng làm đích. Các siêu tham số được tóm tắt trong Bảng 3.2. Với $\gamma = 0.99$, tầm nhìn hiệu dụng $1/(1-\gamma) = 100$ bước, bằng đúng độ dài một episode.

Bảng 3.2: Siêu tham số của DQN.

Tham số	**Giá trị**
Số episode / số bước mỗi episode	1000 / 100
Bộ nhớ phát lại / kích thước batch	10^4 chuyển tiếp / 128
Cập nhật mạng đích	mỗi 5 episode
Hệ số chiết khấu γ	0.99
Tối ưu hóa	Adam [15], tốc độ học 10^{-3}
Thăm dò	$\varepsilon_0 = 1$, nhân 0.99 sau mỗi episode; ngoài ra lấy mẫu theo softmax(Q/τ), $\tau = 5$
Tabu theo thuộc tính	30 bước
Kiến trúc	$d = 64$, $K = 3$ vòng truyền thông điệp

3.5.4 Sử dụng trong ADD*

Sau khi huấn luyện, ADD* xếp hạng ứng viên theo

$$\sigma_{\text{DQN}}(S, i) = Q_\theta\Big((x_S, 0),\ \text{add } i\Big). \tag{3.8}$$

Khoảng cách g được đặt bằng 0 vì ADD* chỉ thêm item: với $g = 0$, mọi phần tăng lợi nhuận đều được tính là phần thưởng ngay, nên Q đo đúng lợi ích của việc thêm. Cần lưu ý rằng DQN học trên quỹ đạo thêm/bỏ của chính nó chứ không phải trên cây ADD*, nên $Q(s, \text{add } i)$ chỉ là một xấp xỉ của đại lượng mà ADD* cần. Đây là một giả thuyết cần kiểm chứng bằng thực nghiệm.

3.6 Phương án 2: bộ xếp hạng học bắt chước tìm kiếm vét cạn

Phương án thứ hai dựa trên một quan sát: với riêng quyết định của ADD*, ta *có thể tính chính xác* ứng viên tốt nhất trên các instance nhỏ bằng tìm kiếm vét cạn. Do đó có thể học hàm xếp hạng bằng học có giám sát, tránh được sự không khớp giữa bài toán huấn luyện và bài toán sử dụng của DQN.

3.6.1 Sinh nhãn bằng tìm kiếm vét cạn

Với một lời giải bộ phận S tại một nút ADD*, định nghĩa giá trị hoàn thiện tối ưu

$$V(S) = \max\Big\{P(T) :\ T \supseteq S,\ W(T) \le C\Big\}, \tag{3.9}$$

tức lợi nhuận tốt nhất đạt được khi chỉ được thêm item vào S. $V(S)$ chính là kết quả của ADD* với $a_{num} = \infty$ (không tính tabu). Nhãn cho ứng viên $i \in FC(S)$ là $V(S \cup \{i\})$; ứng viên tối ưu là $\arg\max_i V(S \cup \{i\})$. V được tính bằng tìm kiếm theo chiều sâu có ghi nhớ (memoization) theo giá trị băm của lời giải, với giới hạn 2×10^5 nút cho mỗi nhãn; nhãn nào vượt giới hạn bị loại.

Dữ liệu được thu thập trong lúc chạy E2LS (với luật tỉ số) để các trạng thái được gán nhãn có cùng phân phối với trạng thái gặp khi tìm kiếm thật. Ở mỗi lần thử bỏ một item, với xác suất 0.05, bộ giải gán nhãn cho các quyết định dọc theo đường đi tối ưu: tại trạng thái hiện tại, thêm các item miễn phí, tính FC và nhãn của mọi ứng viên, lưu mẫu $(x_S, FC, \{V(S \cup \{i\})\}_{i \in FC})$ nếu $|FC| \ge 2$, rồi đi theo ứng viên tối ưu và lặp lại. Việc gán nhãn dùng một bộ sinh số ngẫu nhiên riêng, nên không làm thay đổi quỹ đạo tìm kiếm.

Instance huấn luyện. Để 60 instance chuẩn hoàn toàn không xuất hiện khi huấn luyện, dữ liệu được sinh trên các instance ngẫu nhiên có cùng cấu trúc với bộ chuẩn: mỗi item phủ mỗi element với xác suất α (và phủ ít nhất một element), lợi nhuận và trọng số là số nguyên ngẫu nhiên đều trong $[1, 500]$, và $C = \lfloor \beta \sum_j w_j \rfloor$. Sáu kích thước $m \times n \in \{85\times100,\ 100\times85,\ 100\times100,\ 185\times200,\ 200\times185,\ 200\times200\}$ với hai cặp (α, β) của bộ chuẩn, mỗi cấu hình 4 instance, cho tổng cộng 48 instance. Trên mỗi instance, E2LS chạy 2000 bước và thu tối đa 300 mẫu. 20% số instance (10 instance) được giữ lại làm tập kiểm định; việc chia được thực hiện theo instance chứ không theo mẫu, để đo khả năng tổng quát hóa sang instance chưa gặp.

3.6.2 Mô hình và hàm mất mát

Bộ xếp hạng dùng kiến trúc ở Mục 3.4 (không có đặc trưng khoảng cách g) và cho điểm

$$\sigma_{\text{net}}(S, i) = \underbrace{\log \frac{p_i/\bar{p}}{AW(S,i)/u}}_{\text{luật tỉ số của bài báo}} + \underbrace{\psi\big[h_i^I,\ g\big]}_{\text{hiệu chỉnh học được}}. \tag{3.10}$$

Số hạng thứ nhất bằng $\log R(S, i)$ cộng một hằng số, nên xếp hạng theo nó trùng với xếp hạng của bài báo. Tầng cuối của ψ được khởi tạo bằng 0, nên *một mô hình chưa huấn luyện xếp hạng giống hệt E2LS gốc*, và quá trình học chỉ thay đổi thứ tự ở những chỗ có lợi, chẳng hạn những item mà element của chúng chồng lấn với các item khác. Cách tham số hóa này đóng vai trò như một ràng buộc mềm giữ mô hình gần với heuristic chuyên gia.

Với một mẫu có tập ứng viên FC, gọi $\mathcal{O} \subseteq FC$ là tập ứng viên tối ưu (có thể có nhiều ứng viên cùng giá trị). Hàm mất mát là entropy chéo theo danh sách (listwise) [6]

giữa phân phối softmax của điểm và phân phối đều trên $\mathcal{O}$:

$$\mathcal{L} = -\sum_{i \in FC} \frac{\mathbb{1}[i \in \mathcal{O}]}{|\mathcal{O}|} \log \frac{\exp \sigma_{\text{net}}(S, i)}{\sum_{k \in FC} \exp \sigma_{\text{net}}(S, k)}. \tag{3.11}$$

Ba chỉ số được dùng để đánh giá, cho cả bộ xếp hạng học được và luật tỉ số: (i) **top-1**, tỉ lệ mẫu mà ứng viên xếp đầu là tối ưu; (ii) **hit@2**, tỉ lệ mẫu mà ít nhất một trong hai ứng viên đầu là tối ưu, phù hợp với $a_{num} = 2$; (iii) **regret**, mức mất mát tương đối $(V^\star - V_{\text{top-1}})/V^\star$ khi chọn ứng viên xếp đầu.

Mô hình được huấn luyện 30 epoch bằng Adam với tốc độ học 10^{-3}, batch 32 mẫu (mỗi batch chỉ chứa mẫu của một instance), cắt chuẩn gradient tại 1.0, $d = 64$ và $K = 3$. Checkpoint có hàm mất mát kiểm định nhỏ nhất được giữ lại. Khi sử dụng, một checkpoint duy nhất được dùng cho mọi instance.

3.7 So sánh hai phương án

Bảng 3.3 tóm tắt sự khác nhau giữa hai phương án.

Bảng 3.3: So sánh hai phương án học hàm xếp hạng.

	DQN (Phương án 1)	**Bộ xếp hạng (Phương án 2)**
Tín hiệu học	Cải thiện lời giải tốt nhất (phần thưởng)	Ứng viên tối ưu tính bằng vét cạn (nhãn)
Bài toán được học	Thêm/bỏ một item trên quỹ đạo riêng	Đúng quyết định của ADD*
Phạm vi mô hình	Một mô hình cho mỗi instance	Một mô hình cho mọi instance
Dữ liệu huấn luyện	Chính instance cần giải	Instance sinh ngẫu nhiên, $m, n \leq 200$
Chi phí huấn luyện	Lặp lại cho mỗi instance	Trả một lần
Khi chưa huấn luyện	Tùy ý (tham số khởi tạo ngẫu nhiên)	Trùng luật tỉ số của bài báo
Hạn chế chính	Không khớp giữa huấn luyện và sử dụng	Nhãn chỉ tính được trên instance nhỏ

Về lý thuyết, DQN có ưu điểm là học được giá trị nhiều bước mà không cần nhãn, nhưng chi phí huấn luyện phải trả lại cho từng instance và phải được tính vào thời gian giải nếu so sánh công bằng. Bộ xếp hạng học đúng quyết định mà ADD* cần và được huấn luyện một lần, nhưng nhãn vét cạn chỉ tính được trên instance nhỏ, nên mô hình phải tổng quát hóa từ $m, n \leq 200$ lên tới $m, n = 1000$. Chương 4 trình bày kết quả của bộ giải gốc trên 60 instance chuẩn, kết quả huấn luyện của phương án thứ hai và chi phí suy luận của việc xếp hạng bằng mạng.

Chương 4. Thực nghiệm và đánh giá

Chương này trình bày kết quả thực nghiệm. Mục 4.1 mô tả thiết lập thực nghiệm. Mục 4.2 đánh giá bộ giải E2LS cài đặt lại trên 60 instance chuẩn và so sánh với các công bố. Mục 4.3 phân tích hành vi tìm kiếm của bộ giải. Mục 4.4 trình bày kết quả huấn luyện bộ xếp hạng học bắt chước. Mục 4.5 đo chi phí suy luận khi xếp hạng bằng mạng. Mục 4.6 trình bày quy trình đánh giá các hàm xếp hạng học được trên bộ chuẩn. Mục 4.7 thảo luận kết quả.

4.1 Thiết lập thực nghiệm

Môi trường. Các thực nghiệm chạy trên một máy tính xách tay với CPU Intel Core i7-1255U (2 lõi hiệu năng cao và 8 lõi tiết kiệm điện, 12 luồng), 16 GB RAM, hệ điều hành Ubuntu 22.04, không dùng GPU. Bộ giải được biên dịch bằng g++ 11.4 với `-O3`; mạng nơ-ron dùng PyTorch [19] 2.12 (bản CPU) trên Python 3.10.

Tham số của E2LS. Như bài báo gốc [28]: $r_{num} = a_{num} = 2$ và $t = \sqrt{\max(m, n)}$; không giới hạn số nút của ADD*. Trên bộ chuẩn, bộ giải dùng luật tỉ số của bài báo, chạy *một lần* cho mỗi instance với ngân sách 30 giây trên một lõi CPU (seed 1). Mọi lời giải trả về đều được kiểm tra lại là khả thi bằng bản cài đặt Python độc lập.

Số liệu so sánh. Kết quả của các thuật toán khác được lấy từ các công bố tương ứng: BABC [11], I2PLS [24], KBTS [25], E2LS [28], QL và gbPSO [18], AES-BM [27]. Gọi BKS là lời giải tốt nhất trong các công bố này cho mỗi instance. Khoảng cách tới BKS được tính bằng $(\text{BKS} - P)/\text{BKS}$, với P là lợi nhuận của lời giải tìm được.

Lưu ý về ngân sách. Các công bố tìm kiếm cục bộ dùng ngân sách lớn hơn nhiều: 500 giây cho Set I và 1000 giây cho Set II, và báo cáo kết quả tốt nhất trên 10 lần chạy (E2LS, KBTS) hoặc 100 lần chạy (I2PLS) mỗi instance, trên máy chủ [24, 25, 28]. Luận văn dùng 30 giây và một lần chạy, tức là ngân sách thời gian cho mỗi lần chạy nhỏ hơn 17 đến 33 lần, và không có lợi thế chọn lần chạy tốt nhất. Vì vậy so sánh với các công bố chỉ nhằm kiểm tra rằng bản cài đặt lại hoạt động đúng và ở mức chất lượng hợp lý, chứ không nhằm khẳng định bộ giải vượt các thuật toán khác.

4.2 Kết quả của bộ giải E2LS cài đặt lại

Bảng 4.1 và 4.2 trình bày kết quả trên hai bộ dữ liệu. Bảng 4.3 tóm tắt các con số chính.

Bảng 4.1: Kết quả trên Set I. BABC, QL, gbPSO: số liệu công bố. BKS: lời giải tốt nhất trong các công bố được so sánh (I2PLS, KBTS, E2LS và AES-BM đều đạt BKS trên toàn bộ Set I). Luận văn: một lần chạy 30 giây với luật tỉ số. Δ = Luận văn − BKS. In đậm: bằng BKS.

$m \times n$	BABC	QL	gbPSO	BKS	Luận văn	Δ
$\alpha = 0.10,\ \beta = 0.75$						
85×100	11664	11619	**12045**	12045	**12045**	0
100×85	13251	13251	**13283**	13283	**13283**	0
100×100	13860	**14044**	**14044**	14044	**14044**	0
185×200	13047	13503	**13696**	13696	**13696**	0
200×185	13241	13405	13405	13521	**13521**	0
200×200	11846	12196	**12522**	12522	**12522**	0
285×300	11158	11395	**11568**	11568	**11568**	0
300×285	10428	10952	11335	11563	**11563**	0
300×300	12186	12644	12695	12817	12784	−33
385×400	10085	10022	10483	10600	10490	−110
400×385	10766	11242	**11484**	11484	**11484**	0
400×400	10626	11158	11432	11665	**11665**	0
485×500	10823	10823	11097	11321	11119	−202
500×485	10784	11414	**11771**	11771	11722	−49
500×500	10755	10888	10991	11249	11245	−4
$\alpha = 0.15,\ \beta = 0.85$						
85×100	**12369**	**12369**	**12369**	12369	**12369**	0
100×85	12238	**12479**	12348	12479	**12479**	0
100×100	**13508**	**13508**	**13508**	13508	**13508**	0
185×200	10602	10973	**11298**	11298	**11298**	0
200×185	13829	14044	**14215**	14215	**14215**	0
200×200	11521	11911	**12317**	12317	12113	−204
285×300	10528	11377	11763	11802	11714	−88
300×285	12012	12245	12245	12607	12500	−107
300×300	10382	11282	11425	11585	**11585**	0
385×400	9456	9738	10302	10506	10490	−16
400×385	9649	10419	10526	11209	10758	−451
400×400	9541	10520	10927	11325	**11325**	0
485×500	9333	9649	10117	10220	10120	−100
500×485	9090	9729	10022	10238	9938	−300
500×500	9318	9938	10194	10381	10186	−195

Bảng 4.2: Kết quả trên Set II. KBTS, E2LS: số liệu công bố; BKS: giá trị lớn nhất của hai thuật toán. Luận văn: một lần chạy 30 giây với luật tỉ số. Δ = Luận văn − BKS. In đậm: bằng BKS.

$m \times n$	KBTS	E2LS	BKS	Luận văn	Δ
$\alpha = 0.10,\ \beta = 0.75$					
585×600	**10393**	**10393**	10393	10198	−195
600×585	**9914**	**9914**	9914	9688	−226
600×600	**10524**	**10524**	10524	10391	−133
685×700	**10121**	**10121**	10121	10103	−18
700×685	**9881**	**9881**	9881	9781	−100
700×700	**9786**	**9786**	9786	9709	−77
785×800	**9384**	**9384**	9384	9192	−192
800×785	9837	**9937**	9937	9698	−239
800×800	**9932**	**9932**	9932	9787	−145
885×900	**9318**	**9318**	9318	**9318**	0
900×885	**9725**	**9725**	9725	9594	−131
900×900	**9745**	**9745**	9745	9648	−97
985×1000	9193	**9234**	9234	9101	−133
1000×985	9668	**9689**	9689	**9689**	0
1000×1000	9544	**9551**	9551	9433	−118
$\alpha = 0.15,\ \beta = 0.85$					
585×600	**9256**	**9256**	9256	9119	−137
600×585	**9357**	**9357**	9357	9252	−105
600×600	**9062**	**9062**	9062	8893	−169
685×700	**9176**	**9176**	9176	9064	−112
700×685	**9163**	**9163**	9163	9051	−112
700×700	**9229**	**9229**	9229	9154	−75
785×800	**8746**	**8746**	8746	8505	−241
800×785	**9024**	**9024**	9024	8898	−126
800×800	**9101**	**9101**	9101	8831	−270
885×900	**8425**	**8425**	8425	**8425**	0
900×885	**8620**	**8620**	8620	8342	−278
900×900	**8990**	**8990**	8990	8965	−25
985×1000	8528	**8612**	8612	8426	−186
1000×985	8453	**8455**	8455	8344	−111
1000×1000	8474	**8538**	8538	8248	−290

Bảng 4.3: Tóm tắt kết quả của bộ giải (luật tỉ số, một lần chạy 30 giây) so với BKS.

	Số instance đạt BKS	**Gap trung bình (%)**	**Gap lớn nhất (%)**
Set I, $\alpha = 0.10$, $\beta = 0.75$	10/15	0.24	1.78
Set I, $\alpha = 0.15$, $\beta = 0.85$	7/15	0.88	4.02
Set I (toàn bộ)	17/30	0.56	4.02
Set II, $\alpha = 0.10$, $\beta = 0.75$	2/15	1.22	2.41
Set II, $\alpha = 0.15$, $\beta = 0.85$	1/15	1.68	3.40
Set II (toàn bộ)	3/30	1.45	3.40

Set I. Với chỉ 30 giây, bộ giải đạt BKS trên 17/30 instance, với khoảng cách trung bình 0.56%. So với các thuật toán quần thể, bộ giải không thua BABC hay QL trên instance nào (tốt hơn BABC trên 28 instance và QL trên 26 instance, bằng nhau trên các instance còn lại), và tốt hơn hoặc bằng gbPSO trên 25/30 instance. Các instance chưa đạt BKS tập trung ở kích thước lớn ($m, n \geq 200$) và ở cấu hình dày hơn $\alpha = 0.15$, $\beta = 0.85$, nơi khoảng cách lớn nhất là 451 đơn vị lợi nhuận (4.02%) trên instance 400×385.

Set II. Trên các instance lớn, bộ giải đạt BKS trên 3/30 instance với khoảng cách trung bình 1.45%. Đây là bộ dữ liệu mà ngân sách nhỏ ảnh hưởng nhiều nhất: ở kích thước $m, n \approx 1000$, mỗi bước tìm kiếm tốn khoảng 3 mili giây (Mục 4.3), nên 30 giây chỉ đủ cho vài nghìn bước, trong khi E2LS gốc có 1000 giây cho mỗi lần chạy. Cũng như ở Set I, cấu hình $\alpha = 0.15$, $\beta = 0.85$ khó hơn, với khoảng cách trung bình 1.68% so với 1.22%.

Kết quả trên cho thấy bản cài đặt lại hoạt động đúng: nó tái tạo được BKS trên hơn một nửa Set I trong thời gian ngắn và không có lời giải nào bất khả thi. Khoảng cách còn lại, nhất là trên Set II, là vùng mà một hàm xếp hạng tốt hơn có thể đóng góp, và cũng là trọng tâm của các thực nghiệm tiếp theo.

4.3 Hành vi tìm kiếm của bộ giải

Để hiểu bộ giải dành thời gian vào đâu, luận văn đo số bước tìm kiếm, số lần khởi tạo lại và số nút ADD* trong 5 giây (luật tỉ số, seed 1) trên sáu instance đại diện (Bảng 4.4).

Bảng 4.4: Hành vi tìm kiếm của bộ giải với luật tỉ số trong 5 giây (seed 1). Tỉ lệ khởi tạo lại: số lần khởi tạo lại chia cho số bước.

Instance	Số bước	Bước / giây	Tỉ lệ khởi tạo lại	Số nút ADD*
`sukp_100_100_0.10_0.75`	100 807	20 161	21.0%	320 206
`sukp_100_100_0.15_0.85`	59 085	11 817	34.2%	123 324
`sukp_100_85_0.10_0.75`	107 315	21 463	22.6%	256 479
`sukp_100_85_0.15_0.85`	57 183	11 437	43.8%	70 945
`sukp_1000_1000_0.10_0.75`	1 718	344	4.5%	5 432
`sukp_1000_1000_0.15_0.85`	1 512	302	6.0%	4 555

Có hai nhận xét. Thứ nhất, tốc độ giảm từ 40 đến 60 lần khi kích thước tăng từ 100 lên 1000, vì cả chi phí tính AW và số ứng viên tại mỗi nút ADD* đều tăng theo kích thước. Thứ hai, trên các instance nhỏ, *khởi tạo lại xảy ra rất thường xuyên*: từ 21% đến 44% số bước kết thúc bằng việc mọi láng giềng đều đã nằm trong danh sách tabu và thuật toán phải xây lại lời giải bằng Random_Greedy. Như vậy chất lượng của thủ tục khởi tạo, vốn cũng dùng luật tỉ số, ảnh hưởng đáng kể tới hiệu quả tìm kiếm trên Set I. Trên các instance lớn, tỉ lệ này chỉ khoảng 5%, và phần lớn thời gian được dành cho ADD*.

4.4 Kết quả huấn luyện bộ xếp hạng

Quá trình thu thập nhãn trên 48 instance sinh ngẫu nhiên tạo ra 8 706 mẫu quyết định của ADD*, trong đó 6 950 mẫu thuộc 38 instance huấn luyện và 1 756 mẫu thuộc 10 instance kiểm định. Mỗi instance đóng góp từ 53 đến 261 mẫu. Mỗi mẫu có trung bình 12.2 ứng viên (trung vị 8, nhiều nhất 155), và trung bình 2.13 ứng viên cùng đạt giá trị tối ưu; 53% số mẫu có nhiều hơn một ứng viên tối ưu, cho thấy việc dùng phân phối đích đều trên tập $\mathcal{O}$ trong (3.11) là cần thiết. Toàn bộ quá trình thu thập nhãn và 30 epoch huấn luyện mất dưới 10 phút trên CPU.

Hình 4.1 cho thấy diễn biến của ba chỉ số trên tập kiểm định. Tại epoch 0, mô hình chưa huấn luyện cho kết quả trùng khít với luật tỉ số, đúng như thiết kế ở Mục 3.6. Chỉ sau vài epoch, bộ xếp hạng đã vượt luật tỉ số trên cả ba chỉ số, và các chỉ số ổn định từ khoảng epoch 10. Bảng 4.5 so sánh checkpoint được chọn (epoch 30, có hàm mất mát kiểm định nhỏ nhất) với luật tỉ số.



Hình 4.1: Chỉ số trên tập kiểm định (10 instance chưa gặp) qua các epoch huấn luyện. Epoch 0 là mô hình chưa huấn luyện, trùng với luật tỉ số.

Bảng 4.5: Chất lượng xếp hạng trên tập kiểm định: luật tỉ số của E2LS so với bộ xếp hạng học được (checkpoint epoch 30).

	Top-1 (%)	**Hit@2 (%)**	**Regret (%)**
Luật tỉ số $R(S, i)$	60.9	81.5	0.705
Bộ xếp hạng học được	**71.3**	**87.6**	**0.442**

Kết quả này cho thấy hai điều. Thứ nhất, luật tỉ số của E2LS đã khá tốt nhưng còn xa mức tối ưu: trong gần 40% quyết định, ứng viên mà nó xếp đầu không phải là ứng viên tốt nhất, và trong 18.5% quyết định, cả hai ứng viên được rẽ nhánh đều không tối ưu. Thứ hai, một mạng nơ-ron đồ thị nhỏ có thể học được phần lớn khoảng cách này từ dữ liệu: tỉ lệ chọn đúng tăng thêm 10.4 điểm phần trăm, tỉ lệ bỏ sót ứng viên tối ưu khỏi hai nhánh giảm từ 18.5% xuống 12.4%, và regret giảm 37%. Vì tập kiểm định gồm các instance chưa gặp, mức cải thiện này phản ánh khả năng tổng quát hóa trong cùng phân

phối kích thước ($m, n \leq 200$).

4.5 Chi phí suy luận

Hàm xếp hạng học được phụ thuộc vào lời giải hiện tại nên phải được tính lại tại mỗi nút ADD* có nhiều hơn a_{num} ứng viên. Để đo chi phí này, luận văn chạy 200 bước trên instance lớn nhất `sukp_1000_1000_0.10_0.75`, một lần với luật tỉ số và một lần với mạng (Bảng 4.6).

Bảng 4.6: Chi phí của 200 bước tìm kiếm trên `sukp_1000_1000_0.10_0.75` (CPU).

	Luật tỉ số	**Mạng**
Thời gian cho 200 bước	0.63 s	4.03 s
Số lần gọi mạng	–	409
Thời gian một lần gọi mạng	–	khoảng 9.5 ms
Thời gian trung bình mỗi nút ADD* (luật tỉ số)	khoảng 0.9 ms	–

Mỗi lần gọi mạng tốn khoảng 9.5 mili giây, tức khoảng mười lần chi phí trung bình của một nút ADD* với luật tỉ số, và toàn bộ bước tìm kiếm chậm đi khoảng 6.4 lần. Chi phí này đến từ ba vòng truyền thông điệp trên ma trận kề dày kích thước 1000×1000 và từ việc gọi qua lại giữa C++ và Python. Hệ quả là ở cùng 30 giây, bộ giải dùng mạng chỉ thực hiện được khoảng một phần sáu số bước trên instance lớn. Vì vậy một hàm xếp hạng học được chỉ có lợi trong thực tế nếu chất lượng xếp hạng tốt hơn đủ để bù cho số bước bị mất. Điều này cần được kiểm chứng riêng ở chế độ cùng thời gian, và cũng là động lực cho các hướng giảm chi phí ở Chương 5.

4.6 Đánh giá các hàm xếp hạng học được trên bộ chuẩn

Để đánh giá các hàm xếp hạng học được trên 60 instance chuẩn, luận văn đề xuất quy trình sau, được cài đặt trong `eval_rankers.py` (Phụ lục A). Mỗi cặp (instance, hàm xếp hạng) được chạy với ba seed trong hai chế độ ngân sách:

- **Cùng số bước** (1000 bước tìm kiếm cục bộ): đo *chất lượng xếp hạng*, vì hai bộ giải khám phá cùng số láng giềng và chỉ khác ở thứ tự rẽ nhánh của ADD*.
- **Cùng thời gian** (30 giây): đo *giá trị thực tế*, vì thời gian suy luận của mạng được tính vào ngân sách.

Với DQN, chi phí huấn luyện trên từng instance phải được báo cáo cùng với thời gian giải. Kết quả được tổng hợp trong Bảng 4.7; các ô đánh dấu đỏ sẽ được điền sau khi hoàn tất các lượt chạy.

Bảng 4.7: Luật tỉ số so với các hàm xếp hạng học được trên bộ chuẩn (trung bình 3 seed). Gap: khoảng cách trung bình tới BKS. T/H/B: số instance thắng/hòa/thua so với luật tỉ số.

Ngân sách	Bộ	Gap tới BKS (%)			T/H/B
		Tỉ số	Xếp hạng	DQN	
1000 bước	Set I	...	...	...	...
1000 bước	Set II	...	...	...	...
30 giây	Set I	...	...	...	...
30 giây	Set II	...	...	...	...

4.7 Thảo luận

Bộ giải nền đủ mạnh để làm điểm so sánh. Bản cài đặt C++ của E2LS với luật tỉ số đạt BKS trên 17/30 instance Set I chỉ trong 30 giây, không thua các thuật toán quần thể BABC và QL trên instance nào, và có khoảng cách trung bình 1.45% trên Set II với ngân sách nhỏ hơn 33 lần so với E2LS gốc. Đây là một điểm so sánh chặt chẽ: một hàm xếp hạng học được phải cải thiện một bộ giải vốn đã mạnh, chứ không phải một bộ giải yếu.

Luật tỉ số có dư địa cải thiện, và dư địa đó học được. Trên các quyết định được gán nhãn chính xác, luật tỉ số chọn sai ứng viên tốt nhất ở gần 40% số lần. Bộ xếp hạng học bắt chước giảm tỉ lệ này xuống còn khoảng 29% trên instance chưa gặp, và giảm tỉ lệ bỏ sót ứng viên tối ưu khỏi hai nhánh của ADD* từ 18.5% xuống 12.4%. Nhờ cách tham số hóa dưới dạng hiệu chỉnh của luật tỉ số, mô hình bắt đầu từ đúng chất lượng của thuật toán gốc và chỉ thay đổi những chỗ có lợi.

Câu hỏi còn mở là giá trị thực tế. Có hai trở ngại giữa chất lượng xếp hạng tốt hơn và lời giải tốt hơn trên bộ chuẩn. Thứ nhất là *tổng quát hóa theo kích thước*: bộ xếp hạng được huấn luyện trên $m, n \leq 200$ nhưng Set II có tới $m, n = 1000$, và kinh nghiệm của DeepACO [26] cho thấy mô hình học trên kích thước nhỏ vẫn có ích nhưng mất một phần lợi thế khi áp dụng cho kích thước lớn hơn nhiều. Thứ hai là *chi phí suy luận*: ở cùng thời gian, bộ giải dùng mạng mất khoảng năm phần sáu số bước trên instance lớn (Mục 4.5). Quy trình đánh giá ở Mục 4.6 được thiết kế để tách bạch hai yếu tố này: chế độ cùng số bước đo riêng chất lượng xếp hạng, còn chế độ cùng thời gian đo hiệu quả tổng thể.

Khởi tạo lại là một điểm cắm tiềm năng khác. Trên Set I, từ 21% đến 44% số bước kết thúc bằng khởi tạo lại, và thủ tục khởi tạo cũng dùng luật tỉ số. Một điểm số học được, tính một lần cho mỗi instance và dùng trong cả khởi tạo lẫn ADD*, có thể mang lại lợi ích mà gần như không tốn chi phí suy luận. Hướng này được thảo luận ở Chương 5.

Chương 5. Kết luận và hướng phát triển

5.1 Kết luận

Luận văn nghiên cứu việc kết hợp tìm kiếm cục bộ với học máy cho bài toán cái túi hợp tập, theo nguyên tắc giữ nguyên một thuật toán mạnh và chỉ học lại đúng một thành phần của nó. Thành phần được chọn là bước xếp hạng ứng viên của toán tử ADD* trong E2LS, nơi bài báo gốc dùng tỉ số giá trị trên trọng số, một đánh giá tham lam một bước.

Các kết quả chính của luận văn như sau:

- Luận văn cài đặt lại E2LS bằng C++ với trạng thái tăng dần, danh sách tabu bằng băm Zobrist và giao diện Python cho phép thay hàm xếp hạng. Với một lần chạy 30 giây, bộ giải đạt lời giải tốt nhất đã biết trên 17/30 instance của Set I, không thua các thuật toán quần thể BABC và QL trên instance nào, và có khoảng cách trung bình 1.45% trên Set II dù ngân sách nhỏ hơn 33 lần so với công bố gốc.
- Luận văn mô hình hóa việc chọn nước đi thành một quá trình quyết định Markov với phần thưởng bằng mức cải thiện lời giải tốt nhất, và thiết kế một mạng nơ-ron đồ thị trên đồ thị hai phía item–element có số tham số không phụ thuộc kích thước instance. Mạng này được dùng chung cho hai phương án huấn luyện: DQN trên từng instance và bộ xếp hạng học bắt chước.
- Bộ xếp hạng học bắt chước, được tham số hóa như một hiệu chỉnh của luật tỉ số và huấn luyện từ nhãn tối ưu tính bằng tìm kiếm vét cạn, nâng tỉ lệ chọn đúng ứng viên tối ưu từ 60.9% lên 71.3%, nâng hit@2 từ 81.5% lên 87.6% và giảm regret 37% trên các instance kiểm định chưa gặp. Toàn bộ quá trình huấn luyện mất dưới 10 phút trên CPU.
- Phân tích hành vi tìm kiếm cho thấy khởi tạo lại chiếm 21–44% số bước trên các instance nhỏ, và việc gọi mạng tại mỗi nút ADD* làm bước tìm kiếm chậm đi khoảng 6.4 lần trên instance 1000×1000. Hai quan sát này xác định rõ những nơi mà học máy có thể đóng góp nhiều nhất với chi phí thấp nhất.

Nhìn chung, luận văn cho thấy luật xếp hạng viết tay của E2LS còn dư địa cải thiện đáng kể, và dư địa đó có thể học được bằng một mạng đồ thị nhỏ mà không cần thay đổi khung tìm kiếm. Việc chuyển chất lượng xếp hạng tốt hơn thành lời giải tốt hơn trên bộ chuẩn, khi đã tính cả chi phí suy luận, là bước tiếp theo của nghiên cứu.

5.2 Hạn chế

Luận văn còn một số hạn chế cần được ghi nhận rõ:

- **Ngân sách thực nghiệm nhỏ.** Do chạy trên một máy tính xách tay, mỗi instance chỉ được chạy một lần trong 30 giây, trong khi các công bố dùng 500 đến 1000 giây và báo cáo kết quả tốt nhất của 10 lần chạy trở lên. Vì vậy luận văn không khẳng định được bộ giải đạt chất lượng ngang các công bố ở ngân sách chuẩn, và một lần chạy chưa cho biết độ biến động giữa các seed.
- **Bộ xếp hạng chưa được đánh giá trên bộ chuẩn.** Bộ xếp hạng học được mới được đánh giá về chất lượng xếp hạng trên các instance kiểm định sinh ngẫu nhiên; các lượt so sánh với luật tỉ số trên 60 instance chuẩn (Bảng 4.7) chưa được hoàn tất.
- **Phương án DQN chưa được đánh giá đầy đủ.** Quy trình huấn luyện DQN đã được cài đặt và chạy thông, nhưng việc huấn luyện một mô hình cho mỗi trong 60 instance vượt quá khả năng tính toán của thực nghiệm này. Hơn nữa, vì DQN được huấn luyện trên chính instance cần giải, chi phí huấn luyện phải được tính vào thời gian giải nếu muốn so sánh công bằng với các thuật toán khác.
- **Nhãn chỉ có trên instance nhỏ.** Tìm kiếm vét cạn chỉ khả thi khi $m, n \leq 200$, nên bộ xếp hạng phải tổng quát hóa lên các instance lớn gấp năm lần kích thước huấn luyện.
- **Chi phí suy luận.** Mạng được gọi tại mỗi nút ADD* qua giao diện Python, với ma trận kề dày; chi phí này tăng nhanh theo kích thước instance và làm giảm số bước tìm kiếm trong cùng một khoảng thời gian.

5.3 Hướng phát triển

Từ các kết quả và hạn chế trên, luận văn đề xuất các hướng phát triển sau.

Giảm chi phí suy luận. Có thể chạy mạng trực tiếp trong C++ (chẳng hạn qua LibTorch), dùng ma trận kề thưa thay vì dày (mật độ của bộ chuẩn chỉ 10–15%), và gom nhiều nút ADD* vào một lần gọi mạng. Một hướng triệt để hơn là tách mạng thành hai phần: phần nặng chỉ phụ thuộc instance được tính một lần, còn phần nhẹ phụ thuộc lời giải được tính tại mỗi nút.

Điểm item học được tính một lần cho mỗi instance. DeepACO [26] cho thấy một heuristic học được, tính một lần cho mỗi instance rồi nhân vào heuristic chuyên gia, đã đủ để cải thiện rõ một metaheuristic mà gần như không tốn chi phí suy luận. Với SUKP, có thể dùng $\sigma(S, i) = \log R(S, i) + \beta\, h_\theta(i)$, trong đó $R(S, i)$ giữ phần phụ thuộc trạng thái và $h_\theta(i)$ là điểm của item do mạng tính một lần. Điểm này có thể dùng trong cả ADD* và thủ tục khởi tạo lại RANDOM_GREEDY; trong thực nghiệm của luận văn, khởi tạo lại chiếm một phần đáng kể số bước tìm kiếm trên Set I.

Huấn luyện bằng gradient chính sách. Thay vì nhãn vét cạn, có thể huấn luyện bộ xếp hạng bằng REINFORCE như DeepACO: lấy mẫu nhiều cách hoàn thiện ADD* từ cùng một trạng thái, dùng lợi nhuận cuối làm phần thưởng và trung bình của các mẫu làm đường cơ sở. Cách này không cần nhãn nên huấn luyện được trực tiếp trên instance cỡ Set II, và có thể thưởng theo lợi nhuận *sau vài bước tìm kiếm cục bộ* để mô hình học đúng điều mà bộ giải cần.

Nhiễu loạn có hướng dẫn. Khi mọi láng giềng đều tabu, thay vì khởi tạo lại từ đầu, có thể bỏ những item có điểm học được thấp nhất khỏi lời giải tốt nhất rồi điền lại bằng ADD*, để tìm kiếm tiếp tục ở gần vùng lời giải tốt.

Đánh giá đầy đủ hơn. Cần đánh giá với ngân sách chuẩn của các công bố (500 và 1000 giây, 10 lần chạy), huấn luyện và đánh giá DQN trên toàn bộ bộ chuẩn có tính chi phí huấn luyện, và mở rộng sang bộ Set III với tới 5000 item và sang bài toán BMCP mà E2LS cũng giải được.

Tài liệu tham khảo

[1] Ashwin Arulselvan. A note on the set union knapsack problem. *Discrete Applied Mathematics*, 169:214–218, 2014.

[2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[3] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour d'horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.

[4] Dimitri P. Bertsekas and John N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, 1996.

[5] Edmund K. Burke, Michel Gendreau, Matthew Hyde, Graham Kendall, Gabriela Ochoa, Ender Özcan, and Rong Qu. Hyper-heuristics: a survey of the state of the art. *Journal of the Operational Research Society*, 64(12):1695–1724, 2013.

[6] Zhe Cao, Tao Qin, Tie-Yan Liu, Ming-Feng Tsai, and Hang Li. Learning to rank: From pairwise approach to listwise approach. In *Proceedings of the 24th International Conference on Machine Learning*, pages 129–136, 2007.

[7] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems 32*, 2019.

[8] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1263–1272, 2017.

[9] Fred Glover. Tabu search—part I. *ORSA Journal on Computing*, 1(3):190–206, 1989.

[10] Olivier Goldschmidt, David Nehme, and Gang Yu. Note: On the set-union knapsack problem. *Naval Research Logistics*, 41(6):833–842, 1994.

[11] Yichao He, Haoran Xie, Tak-Lam Wong, and Xizhao Wang. A novel binary artificial bee colony algorithm for the set-union knapsack problem. *Future Generation Computer Systems*, 78:77–86, 2018.

[12] Hans Kellerer, Ulrich Pferschy, and David Pisinger. *Knapsack Problems*. Springer, Berlin, 2004.

[13] Elias B. Khalil, Hanjun Dai, Yuyu Zhang, Bistra Dilkina, and Le Song. Learning combinatorial optimization algorithms over graphs. In *Advances in Neural Information Processing Systems 30*, pages 6348–6358, 2017.

[14] Samir Khuller, Anna Moss, and Joseph Naor. The budgeted maximum coverage problem. *Information Processing Letters*, 70(1):39–45, 1999.

[15] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[16] Wouter Kool, Herke van Hoof, and Max Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations*, 2019.

[17] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[18] Fehmi Burcin Ozsoydan and İlker Gölcük. A reinforcement learning based computational intelligence approach for binary optimization problems: The case of the set-union knapsack problem. *Engineering Applications of Artificial Intelligence*, 118:105688, 2023.

[19] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035, 2019.

[20] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[21] Nguyen Van Thieu and Seyedali Mirjalili. MEALPY: An open-source library for latest meta-heuristic algorithms in Python. *Journal of Systems Architecture*, 139:102871, 2023.

[22] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems 28*, pages 2692–2700, 2015.

[23] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.

[24] Zequn Wei and Jin-Kao Hao. Iterated two-phase local search for the set-union knapsack problem. *Future Generation Computer Systems*, 101:1005–1017, 2019.

[25] Zequn Wei and Jin-Kao Hao. Kernel based tabu search for the set-union knapsack problem. *Expert Systems with Applications*, 165:113802, 2021.

[26] Haoran Ye, Jiarui Wang, Zhiguang Cao, Helan Liang, and Yong Li. DeepACO: Neural-enhanced ant systems for combinatorial optimization. In *Advances in Neural Information Processing Systems 36*, 2023.

[27] Juntao Zhao and Mhand Hifi. An adaptive evolutionary search-based method for efficiently tackling the set-union knapsack problem. *Information Sciences*, 676:120855, 2024.

[28] Wenli Zhu and Liangqing Luo. Efficient and effective local search for the set-union knapsack problem and budgeted maximum coverage problem. *arXiv preprint arXiv:2207.00749*, 2022.

[29] Albert L. Zobrist. A new hashing method with application for game playing. Technical Report 88, Computer Sciences Department, University of Wisconsin, 1970.

Phụ lục A. Hướng dẫn tái lập kết quả

Toàn bộ mã nguồn, dữ liệu và kết quả của luận văn nằm trong kho mã `SUKP_python`. Các lệnh dưới đây được chạy từ thư mục gốc của kho, trong môi trường ảo Python đã cài các gói trong `requirements.txt`.

A.1 Biên dịch bộ giải C++

```
cmake -S . -B build -DPython_EXECUTABLE=$(which python)
cmake --build build -j
```

Lệnh này tạo module `sukp_cpp` chứa bộ giải E2LS (thư mục `cpp_solver/`).

A.2 Huấn luyện bộ xếp hạng

```
python train_ranker.py --samples data/addback_samples.pt --epochs 30 \
    --out checkpoints/addback_ranker.pt
```

Lệnh này sinh 48 instance ngẫu nhiên, thu nhãn bằng tìm kiếm vét cạn, huấn luyện mạng và lưu checkpoint có hàm mất mát kiểm định nhỏ nhất. Nhật ký huấn luyện của luận văn được lưu tại `results_csv/train_ranker.log`.

A.3 Huấn luyện DQN trên một instance

```
python train.py --config helper/config.yaml
```

Instance và siêu tham số được khai báo trong `helper/config.yaml`. Sổ tay `colab_train.ipynb` chạy cùng các lệnh trên GPU của Google Colab cho toàn bộ bộ dữ liệu.

A.4 Đánh giá trên 60 instance chuẩn

```
python eval_rankers.py --iterations 1000 --seeds 1 2 3 \
    --out results_csv/rankers_iter1000.csv
python eval_rankers.py --time 30 --seeds 1 2 3 \
    --out results_csv/rankers_time30.csv
python thesis/make_results.py
```

Hai lệnh đầu chạy bộ giải với luật tỉ số và với bộ xếp hạng học được, ở chế độ cùng số bước và cùng thời gian. Lệnh cuối sinh các bảng và hình của Chương 4 từ các tệp CSV. Để chạy bộ giải trên instance khai báo trong `helper/config.yaml` với một hàm xếp hạng cụ thể, dùng `-ranker ratio` (luật tỉ số của bài báo), `-ranker network` (bộ xếp hạng học bắt chước) hoặc `-ranker dqn` (DQN đã huấn luyện cho instance đó):

```
python play.py --ranker network --time 30
```